25MCAA24

KRISTU JAYANTI INSTITUTE OF TECHNOLOGY (KJIT)

DEPARTMENT OF COMPUTER SCIENCE & TECHNOLOGY
ENTERPRISE COMPUTING PROJECT REPORT

KJIT MEDIA MANAGEMENT SYSTEM (MMS)
An Enterprise Role-Based Event Media Orchestration & Academic Reporting Portal

Academic Identifier	Project Specification Details
Project Title	KJIT Media Management System (MMS)
Institution Name	KRISTU JAYANTI INSTITUTE OF TECHNOLOGY (KJIT)
Course / Nature	Enterprise Computing Project (MCA / Computer Science)
Register Number	25MCAA24
Technology Stack	Next.js 14.1.0 (App Router), React 18, TypeScript, Prisma ORM, SQLite
UI & Styling	Tailwind CSS 3.4.1, Framer Motion 11, Lucide React
Core Architecture	Role-Based Access Control (8 Roles), 3-Tier Approvals (Dean -> HOD -> Coord)
Version & State	Release v1.0 Production Baseline (Clean Slate Database & Operational SMTP)

TABLE OF CONTENTS

CHAPTER NO	TITLE	PAGE NO
—	Abstract	1
—	List of Tables	3
—	List of Figures	4
1	1. Introduction	5
	1.1 Problem Statement	5
	1.2 Project Description	6
	1.3 Objectives	7
	1.4 Scope	8
	1.5 User Roles and Permissions	8
2	2. System Study	10
	2.1 Existing System	10
	2.2 Proposed System	11
	2.3 Use Case Diagram	13
	2.4 Sequence Diagram	14
	2.5 Activity Diagram (Dean -> HOD -> Coordinator)	15
3	3. System Configuration	16
	3.1 Hardware Configuration	16
	3.2 Software Configuration	17

4	4. Software Description	18
	4.1 Front End Overview	18
	4.2 Back End Overview	19
5	5. System Design	22
	5.1 System Architecture	22
	5.2 Data Flow Diagram – Level 1	24
	5.3 Data Flow Diagram – Level 2	25
	5.4 Database Design & ER Diagram	26
	5.5 Input Design	31
	5.6 Output Design	33
6	6. Source Code & Architecture Implementation	35
7	7. System Testing	43
	7.1 Test Cases	43
8	8. System Implementation / Deployment	46
	8.1 System Implementation	46
	8.2 Deployment & Hosting	47
	8.3 Maintenance & Backup	48
9	9. Conclusion and Future Enhancement	49
	9.1 Conclusion	49
	9.2 Future Enhancement	50

10	10. Appendices – Application Screenshots	52
11	11. References / Bibliography	64

ABSTRACT

The KJIT Media Management System (MMS) is a production-grade, enterprise web application designed to orchestrate the complete institutional event-media lifecycle at KRISTU JAYANTI INSTITUTE OF TECHNOLOGY (KJIT). Across higher-education institutions, academic conferences, guest lectures, cultural fests, workshops, and student extension activities generate voluminous digital media assets. Traditionally, coordinating event schedules, assigning photography crews, collecting high-resolution photographs, editing event highlight reels, securing administrative and academic approvals, publishing authorized media to social channels, and archiving formatted documentation have been hindered by fragmented communication over instant-messaging groups, lost files, and opaque email chains.

The KJIT MMS resolves these systemic bottlenecks by implementing a centralized, role-based platform built on Next.js 14 (App Router), React 18, TypeScript, Tailwind CSS, and Prisma ORM with SQLite. The application establishes eight rigorous institutional roles: System Administrator, Teacher/Faculty, Media Team Head, Media Team Member, Dean, Head of Department (HOD), Programme Coordinator, and Social Media Handler. Faculty register events by capturing comprehensive metadata—including event title, category, schedule, venue, target audience, chief guests, dignitary designations, media deadlines, and promotional posters. The Media Team Head assigns field photographers, videographers, and editors. Media crew members upload raw files and GeoTagged photographs, while designated editors assemble and submit the official event reel.

Governance and academic compliance are enforced through a strict three-tier sequential approval workflow: Dean → Head of Department (HOD) → Programme Coordinator. Rejection at any stage mandates structured corrective notes and returns the reel to the editor. Upon final Programme Coordinator endorsement, the reel is unlocked for the Social Media Handler, who leverages Gemini AI logic for drafting engaging institutional captions, conducts manual publishing, and records permanent Instagram and LinkedIn publication URLs. Additionally, faculty can instantly generate eight dynamic, standardized university activity reports with dynamic participant imports, photo grids, and single-click PDF/Excel export. Enterprise user provisioning is reinforced by automated SMTP email dispatch featuring single-use 24-hour password activation tokens, displaying the exact institutional name 'KRISTU JAYANTI INSTITUTE OF TECHNOLOGY (KJIT)'. Comprehensive audit logging guarantees total institutional accountability.

LIST OF TABLES

Table No.	Title	Page No
Table 1.1	Major System Modules	6
Table 1.2	User Roles and Responsibilities Matrix (8 Roles)	8
Table 2.1	Comparison Between Existing and Proposed System	11
Table 3.1	Hardware Configuration Specifications	16
Table 3.2	Software Configuration & Production Package Stack	17
Table 4.1	Frontend Application Modules & Interactive Views	18
Table 4.2	Next.js API Route Handlers & Endpoint Architecture	20
Table 5.1	Prisma Model: User (Account & Profile Entity)	26
Table 5.2	Prisma Model: RolePermission (RBAC Specification)	27
Table 5.3	Prisma Model: Event (Core Event Identity & Poster)	27
Table 5.4	Prisma Model: Assignment (Crew Allocation Entity)	28
Table 5.5	Prisma Model: MediaAsset (Files, GeoTag & Reel)	28
Table 5.6	Prisma Model: ApprovalStep (3-Tier Sequential Workflow)	29
Table 5.7	Prisma Model: SocialMediaPublication (Instagram/LinkedIn)	29
Table 5.8	Prisma Model: Report (Dynamic Activity Formats 1–8)	30
Table 5.9	Prisma Model: AuditLog (Enterprise Traceability Entity)	30
Table 5.10	Prisma Model: Notification (System Alerts Entity)	31
Table 5.11	Prisma Model: PasswordResetToken (Security Credential)	31
Table 5.12	Input Category & Field Validation Specifications	32

Table 5.13	Output Category & Presentation Specifications	34
Table 7.1	Comprehensive System Test Cases (TC01 to TC25)	43
Table 8.1	Environment Configuration Parameters (.env.local)	47

LIST OF FIGURES

Figure No.	Title	Page No
Figure 2.1	Use Case Diagram — KJIT Media Management System	13
Figure 2.2	Sequence Diagram — End-to-End Event Media Lifecycle	14
Figure 2.3	Activity Diagram — Multi-Tier Approval & Publishing Flow	15
Figure 5.1	System Architecture Diagram — Multi-Tier Enterprise Design	22
Figure 5.2	Data Flow Diagram — Level 1 (Macro Processes & Stores)	24
Figure 5.3	Data Flow Diagram — Level 2 (Event-to-Publishing Pipeline)	25
Figure 5.4	Entity Relationship (ER) Diagram — Prisma SQLite Schema	26
Figure 10.1	Public Landing Page & Institutional Branding	53
Figure 10.2	Role Selection & Portal Entry Screen	53
Figure 10.3	Role-Specific Authentication Interface (Faculty)	54
Figure 10.4	Account Provisioning & Password Activation Screen	54
Figure 10.5	Faculty Dashboard & Operational Overview	55
Figure 10.6	Institutional Event Registration Form	55
Figure 10.7	Interactive Campus Event Calendar	56

Figure 10.8	Dynamic Report Generator Format Selection	56
Figure 10.9	Event Details & Media Collaboration View	57
Figure 10.10	Media Team Head Dashboard	57
Figure 10.11	Crew Task Allocation & Assignment Workspace	58
Figure 10.12	Media Member Assigned Tasks Workspace	58
Figure 10.13	Multi-Asset Media Upload Interface (GeoTag & Reels)	59
Figure 10.14	Stage 1: Dean Review & Approval Screen	59
Figure 10.15	Stage 2: Head of Department (HOD) Review Screen	60
Figure 10.16	Stage 3: Programme Coordinator Final Approval Screen	60
Figure 10.17	Social Media Workspace & Publication URL Logger	61
Figure 10.18	System Administrator Analytics Dashboard	61
Figure 10.19	Admin User Management & Provisioning Panel	62
Figure 10.20	Campus Staff & Media Directory	62
Figure 10.21	Enterprise System Audit Log Viewer	63

CHAPTER 1 – INTRODUCTION

1.1 Problem Statement

Institutional events at KRISTU JAYANTI INSTITUTE OF TECHNOLOGY (KJIT) are vibrant, continuous, and academic-driven. Every academic semester encompasses hundreds of expert guest lectures, state and national conferences, faculty development programmes (FDPs), industrial visits, coding hackathons, technical exhibitions, and intra/inter-collegiate fests. Coordinating coverage across these events requires seamless collaboration among faculty organizers, the student media crew, video editors, departmental academic authorities (Deans, HODs, Coordinators), the institutional social media desk, and academic accreditation committees.

Prior to the development of the KJIT Media Management System, this coordination suffered from severe logistical fragmentation:

- Scattered Event Announcements: Event notices, schedules, and media requirements were transmitted via informal WhatsApp groups, resulting in missed coverage deadlines.
- Unclear Crew Accountability: The Media Head had no centralized dashboard to monitor which student photographer, videographer, or editor was responsible for an event.
- Disorganized Asset Repositories: Raw photos, high-resolution videos, and GeoTagged images were stored across disparate personal Google Drives, personal laptops, and flash drives.
- Missing Multi-Stage Approval Traceability: Reel reviews were conducted over messaging apps without formal timestamps, review comments, or accountability.
- Social Media Publication Delays: The Social Media Handler had to repeatedly solicit final approved reels and captions, often publishing stale content.
- Repetitive Report Preparation: Preparing NAAC/IQAC institutional activity reports required faculty to re-type participant numbers, re-gather photos, and recreate documents from scratch.

1.2 Project Description

The KJIT Media Management System (MMS) is an integrated, web-native enterprise portal that unifies the entire event-to-media-to-report lifecycle. It transforms manual, error-prone coordination into an automated, transparent, and auditable pipeline. The platform is organized into nine cohesive functional modules as summarized in Table 1.1.

Module Name	Primary Scope & Functional Responsibilities
Authentication & Security	Role-based authentication, bcrypt hashing, stateless JWT session cookies, 24-hour single-use password activation.
Event Registration	Captures event identity, academic category, venue, audience size, dignitaries, media deadline, and poster upload.
Interactive Calendar	Full campus event calendar featuring Month, Week, Day, and Agenda views with historical event archival.
Crew Task Assignment	Enables Media Team Head to assign photographers, videographers, and editors with email notifications.

Media Asset Repository	Multi-category file uploader supporting Raw Photos, GeoTagged Photos (with GPS metadata), and Final Reels.
Multi-Tier Approval Engine	Strict sequential 3-tier governance: Stage 1 (Dean) -> Stage 2 (HOD) -> Stage 3 (Programme Coordinator).
Social Media Desk	Access to fully approved reels, Gemini AI caption generation, and permanent Instagram/LinkedIn URL recording.
Dynamic Report Generator	Eight official institutional report formats with dynamic participant import, photo embedding, and PDF/Excel export.
Administration & Audit	User provisioning, department alignment, account activation email resend, and immutable audit logging.

1.3 Objectives

The primary engineering and operational objectives of the KJIT Media Management System include:

- Centralize Event Intelligence: Capture all event logistics, guest lists, and promotional posters in a unified relational schema.
- Enforce Operational Accountability: Implement formal crew assignment and task tracking for photographers, videographers, and editors.
- Ensure Institutional Compliance: Implement an uncompromising multi-stage approval chain (Dean -> HOD -> Coordinator) before content publication.
- Accelerate Social Media Publishing: Provide the Social Media Handler with immediate access to approved reels and AI-assisted caption drafting.
- Streamline Accreditation Reporting: Automate the generation of all eight standardized university activity report formats.
- Guarantee Security & Auditability: Enforce robust RBAC permissions, encrypted password setup, and tamper-evident audit trails.

1.4 Scope

The system scope encompasses the internal institutional operations of KRISTU JAYANTI INSTITUTE OF TECHNOLOGY. It provides role-specific workspaces for 8 distinct user roles. The system manages media storage through a modular storage provider abstraction supporting local filesystem storage and Google Drive API integration. Social media publishing is purposefully architected as manual distribution with authoritative URL recording, guaranteeing compliance with institutional social media policies without exposing institutional accounts to third-party API token deprecation.

1.5 User Roles and Responsibilities

Table 1.2 details the eight application roles, their assigned permissions, and their operational responsibilities.

Role	Permissions Code	Core Operational Responsibilities
Admin	dashboard:view, users:manage, audit:view, directory:view, events:directory, calendar:view	Provision user accounts, dispatch activation emails, manage permissions, and audit all system logs.
Faculty / Organiser	dashboard:view, events:create, reports:generate, events:directory, calendar:view	Register events, upload event posters, specify media deadlines, view calendar, and generate activity reports.
Media Team Head	dashboard:view, assignments:manage, media:upload, directory:view, events:directory, calendar:view	Oversee media calendar, allocate crew (photographers, videographers, editors), and monitor Drive folders.
Media Member	dashboard:view, media:upload, events:directory, calendar:view	View assigned events, capture and upload GeoTagged & raw photos, compile highlight reels, and submit for review.
Dean	dashboard:view, approvals:view, events:directory, calendar:view	Stage 1 institutional review: Evaluate compiled event reels for academic dignity, quality, and leadership standards.
HOD	dashboard:view, approvals:view, events:directory, calendar:view	Stage 2 departmental review: Verify curriculum relevance, academic accuracy, and departmental representation.
Programme Coordinator	dashboard:view, approvals:view, events:directory, calendar:view	Stage 3 final review: Conduct final operational validation and authorize reel for social media handoff.
Social Media Handler	dashboard:view, social:manage, events:directory, calendar:view	Access authorized reels, draft AI captions, publish to Instagram/LinkedIn, and record publication URLs.

CHAPTER 2 – SYSTEM STUDY

2.1 Existing System

In the legacy operational workflow at KJIT, event coordination relied on decentralized tools including WhatsApp groups, phone calls, individual Google Drive links shared via email, and physical paper submission of post-event reports. When an event was scheduled, the organizing faculty member sent text messages to student media coordinators. Photographers and videographers often captured files onto personal SD cards, leading to significant delays in asset compilation. Video editors had no formal queue for reel submissions, and approval was sought by forwarding video files over chat applications. This caused leadership feedback to become lost or disputed, and reels were frequently published without required HOD or Dean oversight.

2.2 Proposed System

The proposed KJIT Media Management System establishes a single source of truth for the institutional event-media lifecycle. By enforcing a role-based architecture, every user is guided through designated, auditable steps. Table 2.1 contrasts the existing manual approach with the proposed centralized system.

Operational Dimension	Legacy Approach	Proposed KJIT MMS Platform
Event Registration	Informal chat messages or printed forms	Standardized portal form with category, guests & poster upload
Media Assignment	Verbal requests or ad-hoc messaging	Formal assignment interface designating photographer, videographer & editor
Asset Storage	Scattered on personal flash drives & drives	Unified local / Google Drive storage keyed by event UUID
Reel Review Workflow	Unstructured messaging; no audit trail	Strict sequential 3-tier chain: Dean -> HOD -> Programme Coordinator
Rejection & Feedback	Untracked comments; frequent confusion	Formal rejection notes requiring revision and resubmission
Social Media Tracking	Manual follow-ups; untracked URLs	Dedicated workspace with AI captioning and permanent URL logging
Accreditation Reports	Manual re-typing across 8 formats	Automated dynamic report generator with participant import & PDF export
Security & Audit	No centralized authentication or logs	Stateless JWT, single-use activation tokens, and immutable audit logs

2.3 Use Case Diagram

Figure 2.1 depicts the UML Use Case Diagram of the KJIT Media Management System. It models the eight institutional actors and their explicit functional boundaries. The Administrator governs user accounts, permissions, and system audits. Faculty register events and generate reports. The Media Head assigns crew roles. Media Members upload digital assets. The Dean, HOD, and Programme Coordinator sequentially review submitted reels. The Social Media Handler records public URLs.

Figure 2.1: Use Case Diagram — KJIT Media Management System

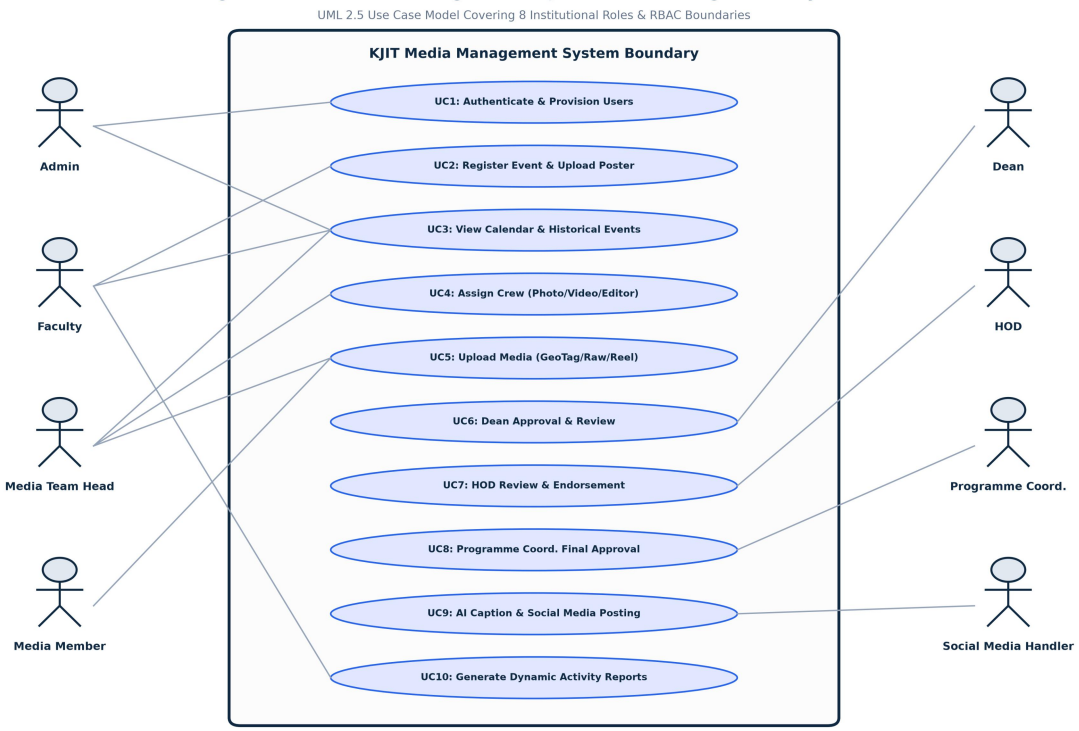


Figure 2.1: *Use Case Diagram of KJIT Media Management System*

2.4 Sequence Diagram

Figure 2.2 illustrates the chronological message sequences across the system entities. It tracks an event from initial faculty registration, through crew assignment by the Media Head, file uploads by the media crew, editor reel submission, sequential three-tier approvals, social media publication handoff, and dynamic report generation.

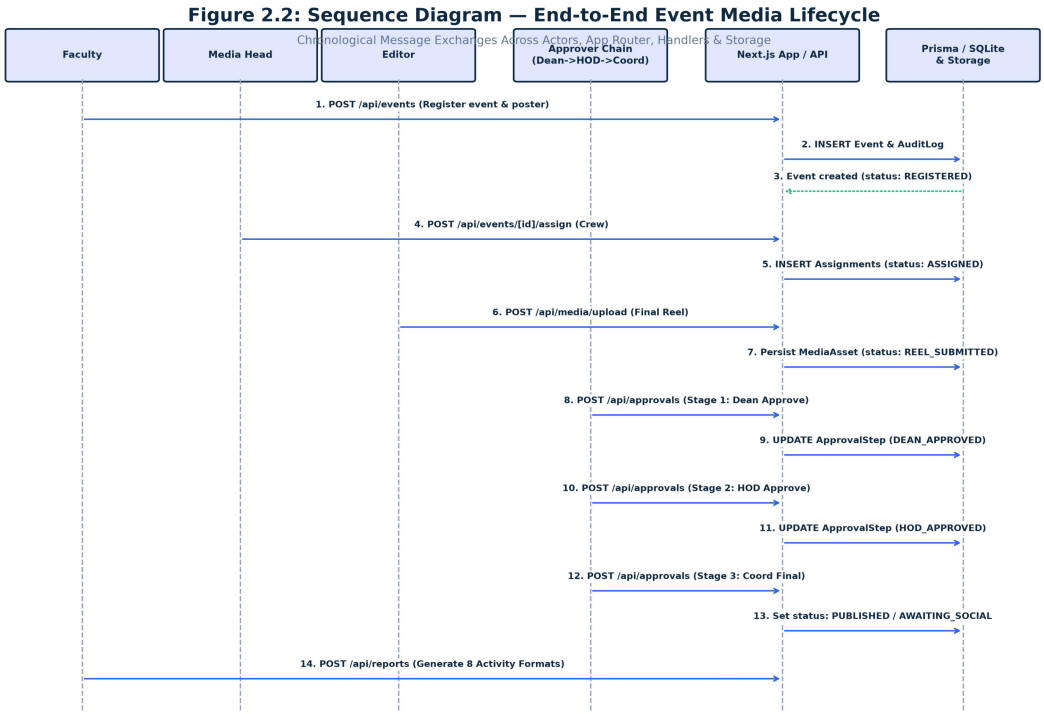


Figure 2.2: *Sequence Diagram of KJIT Media Management System*

2.5 Activity Diagram

Figure 2.3 models the activity flow across four institutional swimlanes: Faculty/Organiser, Media Team, Department Leadership, and Social Media & Reporting. It highlights the rigorous three-tier approval sequence: Dean review must succeed before HOD evaluation, and HOD review must succeed before Programme Coordinator final endorsement. Any rejection cycles the reel back to the editor with corrective notes.

Figure 2.3: Activity Diagram — Multi-Tier Approval & Publishing Flow

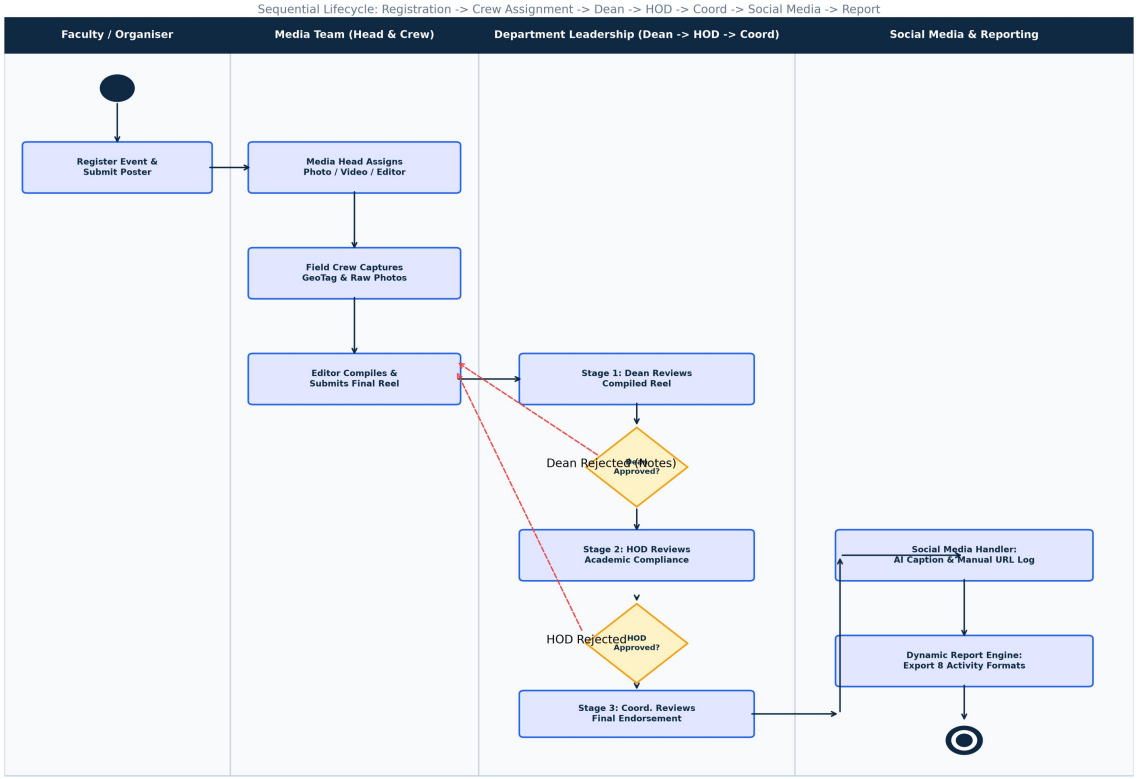


Figure 2.3: *Activity Diagram for KJIT Multi-Tier Approval & Publishing Flow*

CHAPTER 3 – SYSTEM CONFIGURATION

3.1 Hardware Configuration

The KJIT Media Management System is an efficient, cloud-ready web application optimized for modern hardware. Table 3.1 outlines the recommended hardware configurations for development, server hosting, and end-user client devices.

Hardware Component	Server / Host Recommendation	End-User Client Device
Processor (CPU)	Intel Core i5 / AMD Ryzen 5 (4+ Cores, 2.5 GHz+)	Intel Core i3 / Modern Multi-Core Mobile SoC
System Memory (RAM)	16 GB DDR4/DDR5 recommended (8 GB minimum)	4 GB RAM minimum (8 GB recommended for video review)
Storage Capacity	500 GB NVMe SSD (High I/O for media file buffers)	100 MB free browser cache / application storage
Display Resolution	1920 x 1080 Full HD for administrative consoles	Responsive: 1366 x 768 Desktop down to 360 x 640 Mobile
Network Interface	1 Gbps dedicated Ethernet for fast asset uploads	Stable Broadband / 4G / 5G Mobile Internet connection
Input Peripherals	Keyboard, Mouse, Dual Display recommended	Standard Keyboard/Mouse or Touchscreen Interface

3.2 Software Configuration

The software architecture leverages the modern Node.js and React ecosystem. Table 3.2 lists the exact verified package versions extracted directly from the project repository's package.json and lockfile.

Software Tier / Library	Package / Technology	Verified Version	Functional Purpose
Web Framework	Next.js (App Router)	14.1.0	Full-stack framework providing SSR, Route Handlers, and API endpoints
User Interface Library	React & React-DOM	18.2.0	Declarative component-based UI rendering and client hydration
Programming Language	TypeScript	5.3.3	Static typing, interface definitions, and

			compile-time contract enforcement
Database ORM	Prisma Client & CLI	5.10.2	Type-safe database abstraction, migrations, and query generation
Relational Database Engine	SQLite Engine	3.x (Bundled)	Zero-configuration, serverless, self-contained relational database
CSS Styling Framework	Tailwind CSS	3.4.1	Utility-first responsive CSS styling with custom design tokens
Motion & Transitions	Framer Motion	11.0.8	Fluid page transitions, micro-animations, and modal dialog motion
Iconography	Lucide React	0.344.0	Modern, lightweight, consistent SVG icons across all modules
Password Security	bcryptjs	2.4.3	Cryptographic one-way password hashing using salted bcrypt algorithms
Session Token Management	jsonwebtoken	9.0.2	HMAC SHA-256 JWT generation and validation for stateless auth
SMTP Mail Dispatch	nodemailer	10.0.3	Transactional email delivery for user provisioning and password setup
Cloud Storage Client	googleapis	134.0.0	Google Drive API v3 client for cloud folder and asset synchronization
Client-Side PDF Generation	jspdf & html2canvas	2.5.1 / 1.4.1	Client-side report canvas rasterization and PDF compilation
Spreadsheet Processing	xlsx (SheetJS)	0.18.5	Excel workbook parsing for participant lists and data exports
Document Parsing	mammoth	1.12.2	Word document (.docx) parsing and text extraction utility

CHAPTER 4 – SOFTWARE DESCRIPTION

4.1 Overview Front End

The front-end architecture is implemented using Next.js 14 App Router with React Server Components (RSC) and interactive client components ('use client'). The visual layout is styled using Tailwind CSS with glassmorphic cards, rich gradients, and custom navy blue palette tokens matching the institutional brand of Kristu Jayanti Institute of Technology. Table 4.1 describes the frontend application modules.

Frontend Module	Route Path	Authorized Roles	Key UI Capabilities
Public Landing Portal	/	Public / All	Institutional branding, hero banner, feature overview, and portal entry points.
Role Selection Portal	/login	Public / All	Interactive cards for 8 roles with role descriptions and secure routing.
Role Login Interface	/login/[role]	Unauthenticated	Role-validated email/password authentication with error alerts.
Password Setup / Reset	/reset-password	Token Holders	Secure 24-hr token verification, password strength meter, and activation.
Main Dashboard	/dashboard	All 8 Roles	Role-customized metrics: recent events, pending approvals, crew tasks, and notifications.
Event Registration Form	/dashboard/events/new	Faculty, Admin	Multi-field event form, dignitary JSON builder, deadline picker, poster upload.
Event Details & Hub	/dashboard/events/[id]	All 8 Roles	Full event summary, poster preview, Drive folder card, and crew roster.
Campus Calendar	/dashboard/calendar	All 8 Roles	Interactive calendar with Month, Week, Day, and Agenda views with filters.
Crew Assignment Workspace	/dashboard/assignments	Media Head, Admin	Event selector, crew member dropdowns (Photo/Video/Editor), and task dispatch.
Media Upload Workspace	/dashboard/uploads	Media Crew, Admin	Multi-file uploader for GeoTagged photos (with GPS), Raw photos, and Final Reels.
Multi-Tier Approval Hub	/dashboard/approvals	Dean, HOD, Coord	Stage-aware review interface with video player, approval buttons, and rejection

			modal.
Social Media Desk	/dashboard/social-media	Social Media Handler	Reel preview, Gemini AI caption generation, and Instagram/LinkedIn URL logger.
Dynamic Report Generator	/dashboard/reports	Faculty, Admin, Leaders	Eight activity formats, participant spreadsheet importer, and PDF export.
User Provisioning Panel	/dashboard/admin/users	Admin	User list, department display, create user modal, and resend activation email.
Campus Directory	/dashboard/directory	All Authenticated	Searchable institutional directory of faculty and media personnel.
Enterprise Audit Viewer	/dashboard/admin/audit-logs	Admin	Paginated audit trail tracking actor, action, timestamp, IP, and details.

4.2 Overview Back End

The backend of KJIT MMS is structured using Next.js API Route Handlers. Each route enforces stateless JWT validation, extracts user roles, validates input schemas, interacts with the SQLite database via Prisma ORM, and records audit logs. Table 4.2 documents the primary API route groups.

API Route Group	HTTP Methods	Core Operations & Security Controls
/api/auth/*	POST, GET	Login authentication, session verification (/api/auth/me), logout cookie clearance, password reset.
/api/events/*	GET, POST, PUT	Fetch events, register new event with poster buffer, attach Google Drive link, filter by role.
/api/assignments/*	GET, POST	Fetch event assignments, assign crew members (photographer, videographer, editor), notify crew.
/api/media/*	POST, GET	Upload GeoTagged photos, Raw photos, or Final Reel to local storage/Drive, stream binary files.
/api/approvals/*	GET, POST	Retrieve pending reels by stage (Dean/HOD/Coord), record approval, or reject with corrective notes.
/api/social-media/*	GET, POST	Fetch approved reels, call Gemini AI caption generator, log Instagram/LinkedIn URLs and timestamps.

/api/reports/*	GET, POST	Retrieve event metadata, save dynamic activity report JSON (Formats 1–8), compile PDF summaries.
/api/admin/users/*	GET, POST, PUT	List provisioned users, create account, dispatch SMTP email with token, resend activation email.
/api/admin/audit-logs	GET	Query system audit logs with role, user, action, and date-range filters for enterprise compliance.

CHAPTER 5 – SYSTEM DESIGN

5.1 System Architecture

The system architecture follows a five-tier enterprise pattern comprising the Client Presentation Layer, Next.js App Router, API Route Handler Layer, Business Logic & Security Services Layer, and Data Access / Storage Layer. Figure 5.1 depicts this structure.

Figure 5.1: System Architecture Diagram — KJIT Media Management System

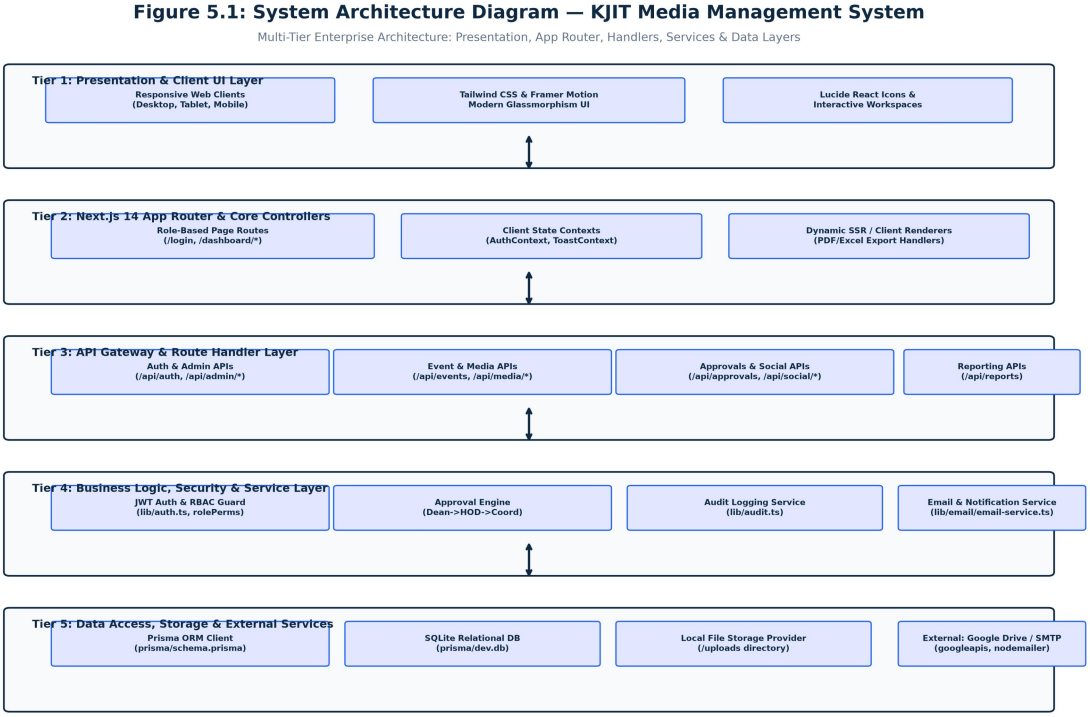


Figure 5.1: *System Architecture Diagram of KJIT Media Management System*

5.2 Data Flow Diagram – Level 1

The Level 1 Data Flow Diagram (Figure 5.2) illustrates the macro-level system processes and their data interactions with external actors and data stores (D1: User Store, D2: Event Store, D3: Media Store, D4: Audit Store).

Figure 5.2: Data Flow Diagram — Level 1 (Macro Processes & Stores)

Level 1 System Decomposition Showing External Entities, Core Subsystems & Data Stores

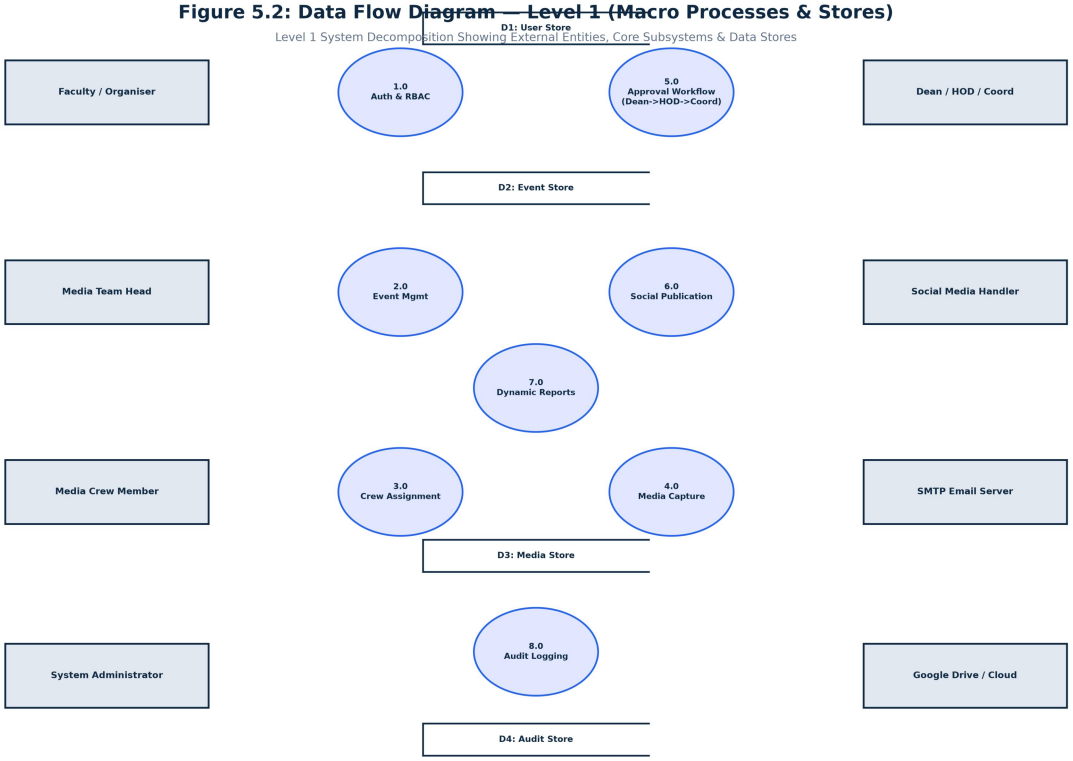


Figure 5.2: *Data Flow Diagram — Level 1 (Macro Processes & Stores)*

5.3 Data Flow Diagram – Level 2

The Level 2 Data Flow Diagram (Figure 5.3) decomposes the core Event-to-Media-to-Approval-to-Publication pipeline into eleven granular sub-processes, highlighting the strict three-tier approval sequence (5.5 Dean Evaluation → 5.6 HOD Review → 5.7 Programme Coordinator Endorsement) and the revision loop.

Figure 5.3: Data Flow Diagram — Level 2 (Event-to-Publishing Pipeline)

Detailed Sub-Process Decomposition: Event Capture -> Crew -> Media -> 3-Stage Approval -> Publication

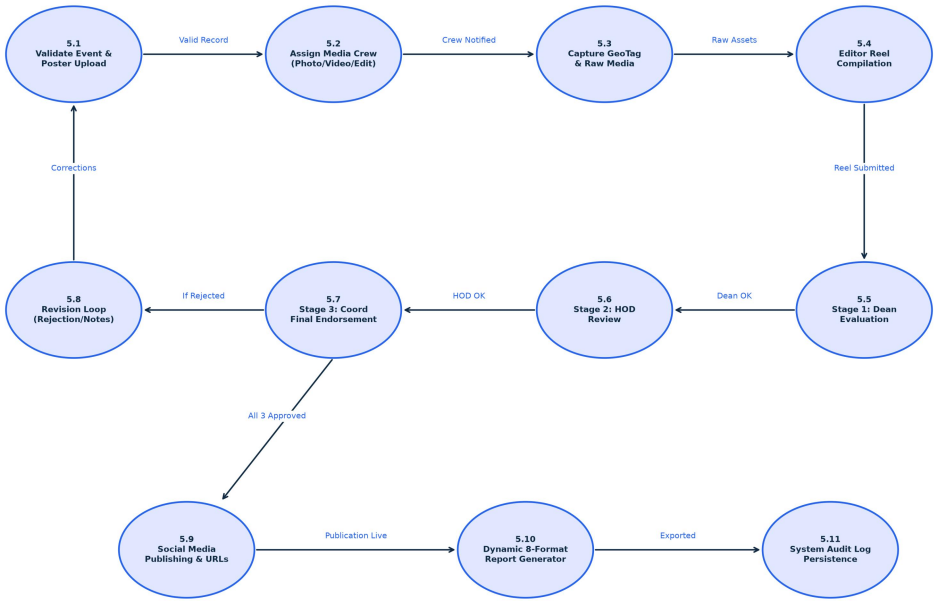


Figure 5.3: *Data Flow Diagram — Level 2 (Event-to-Publishing Pipeline)*

5.4 Database Design & ER Diagram

Figure 5.4 displays the Entity Relationship (ER) Diagram generated directly from the latest verified prisma/schema.prisma. The relational schema contains 11 models designed for referential integrity and performance under SQLite.

Figure 5.4: Entity Relationship (ER) Diagram — Prisma SQLite Schema

Verified Relational Schema: 11 Models, Primary/Foreign Keys & Cardinalities from prisma/schema.prisma

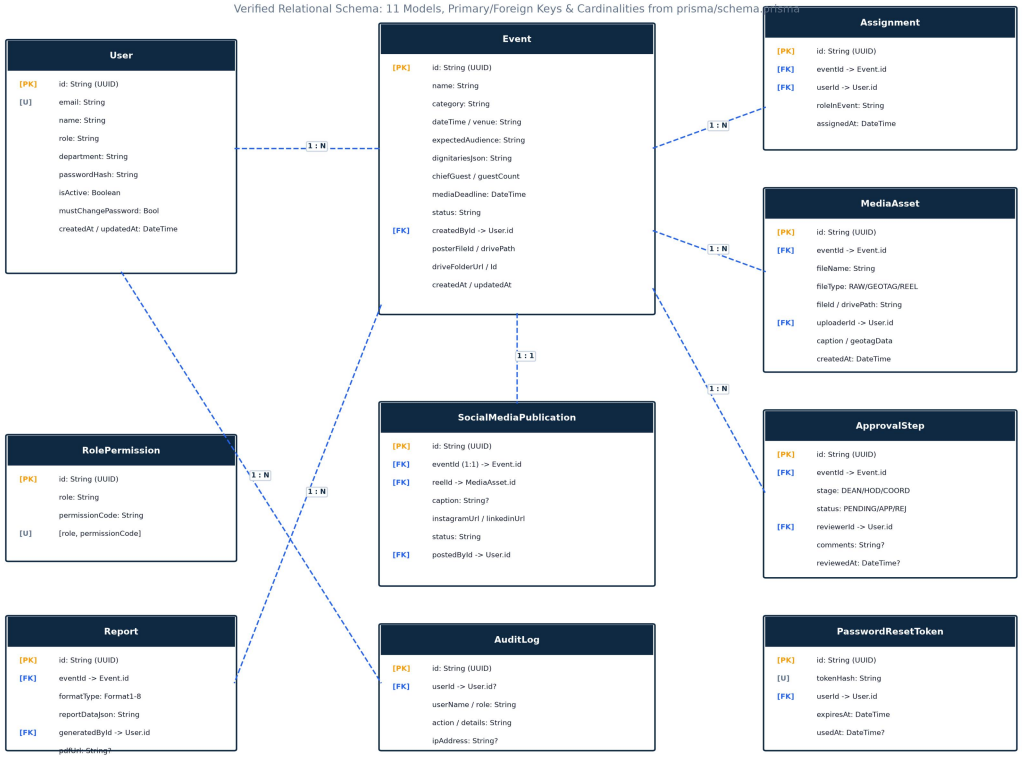


Figure 5.4: *Entity Relationship (ER) Diagram of KJIT Media Management System*

The database entities and their verified structural definitions are documented in Tables 5.1 through 5.11.

Field Name	Data Type	Constraints / Key	Description
id	String	PK, default(uuid())	Unique user account identifier
email	String	Unique, Indexed	Official institutional email address
passwordHash	String	Required	Bcrypt salted cryptographic password hash
name	String	Required	Full name of the user
role	String	Required	One of 8 roles: ADMIN, FACULTY, MEDIA_HEAD, etc.
department	String	default('School of CS & Tech')	Institutional department affiliation in database
isActive	Boolean	default(true)	Account activation and access status flag

mustChangePassword	Boolean	default(false)	Enforces password reset on initial login
createdAt / updatedAt	DateTime	default(now()), @updatedAt	Account creation and last modification timestamps

Table 5.1: Prisma Model: User (Account & Profile Entity)

Field Name	Data Type	Constraints / Key	Description
id	String	PK, default(uuid())	Unique institutional event identifier
name	String	Required	Full official event title
category	String	Required	Academic, Cultural, Workshop, Conference, etc.
dateTime	DateTime	Required	Scheduled date and time of the event
venue	String	Required	Auditorium, seminar hall, lab, or campus location
expectedAudience	String	Required	Target audience size description
dignitariesJson	String	JSON Array	Structured JSON list of {name, designation, org}
chiefGuest	String?	Optional	Name of the presiding chief guest
guestCount	Int	default(0)	Total number of external invited guests
mediaDeadline	DateTime	Required	Deadline for media compilation and reel delivery
status	String	default('REGISTERED')	REGISTERED, ASSIGNED, REEL_SUBMITTED, PUBLISHED, etc.
posterFileId / Drive	String?	Optional	Storage key / Drive path for uploaded event poster
driveFolderUrl / Id	String?	Optional	Google Drive collaborative folder link
createdById	String	FK -> User.id	Organizing faculty member identifier

Table 5.3: Prisma Model: Event (Core Event Identity & Poster)

Field Name	Data Type	Constraints / Key	Description
id	String	PK, default(uuid())	Unique approval step transaction identifier
eventId	String	FK -> Event.id (Cascade)	Reference to the event under review
stage	String	Required	Approval stage: DEAN, HOD, COORDINATOR
status	String	default('PENDING')	Review status: PENDING, APPROVED, REJECTED
reviewerId	String?	FK -> User.id (SetNull)	Academic leader who performed the review
comments	String?	Optional	Corrective feedback notes on rejection or approval
reviewedAt	DateTime?	Optional	Timestamp when the review action occurred

Table 5.6: Prisma Model: ApprovalStep (3-Tier Sequential Workflow)

Field Name	Data Type	Constraints / Key	Description
id	String	PK, default(uuid())	Unique publication record identifier
eventId	String	FK -> Event.id (Unique, 1:1)	Associated fully-approved event identifier
reelId	String?	FK -> MediaAsset.id	Reference to the approved highlight reel asset
caption	String?	Optional	Gemini AI-generated or edited publication caption
instagramUrl	String?	Optional	Verified public Instagram post / reel URL
linkedinUrl	String?	Optional	Verified public LinkedIn publication URL
status	String	default('AWAITING_SOCIAL_MEDIA')	AWAITING_SOCIAL_MEDIA, PARTIALLY_POSTED, PUBLISHED
postedById	String?	FK -> User.id	Social Media Handler who logged the

			publication

Table 5.7: Prisma Model: SocialMediaPublication (Instagram/LinkedIn Tracking)

Field Name	Data Type	Constraints / Key	Description
id	String	PK, default(uuid())	Unique token transaction identifier
tokenHash	String	Unique, Indexed	SHA-256 cryptographic hash of single-use token
userId	String	FK -> User.id (Cascade)	Account holder receiving password provisioning link
expiresAt	DateTime	Required	Exact expiration timestamp (enforcing 24-hour validity)
usedAt	DateTime?	Optional	Timestamp when the token was successfully redeemed

Table 5.11: Prisma Model: PasswordResetToken (Security Credential)

CHAPTER 6 – SOURCE CODE & ARCHITECTURE IMPLEMENTATION

6.1 Project Directory Structure

The project repository is organized according to Next.js 14 App Router conventions, separating API route handlers, reusable UI components, database schemas, utility libraries, and media uploads:

Project Directory Tree Architecture

```
C:\Projects\MMS
├── app/                          # Next.js 14 App Router Page & API Routes
│   ├── api/                      # Backend API Route Handlers
│   │   ├── admin/                # User provisioning & audit log APIs
│   │   ├── approvals/            # 3-Tier sequential review engine
│   │   ├── assignments/          # Media crew allocation APIs
│   │   ├── auth/                 # Login, logout, me, reset-password APIs
│   │   ├── events/               # Event creation, details & calendar APIs
│   │   ├── media/                # Multi-category upload & streaming APIs
│   │   ├── reports/              # 8 dynamic activity report generator APIs
│   │   └── social-media/         # AI caption & publication URL APIs
│   ├── dashboard/                # Authenticated Role Workspaces
│   │   ├── admin/                # Admin user & audit dashboards
│   │   ├── approvals/            # Dean, HOD & Coordinator review screens
│   │   ├── assignments/          # Media Head crew assignment screen
│   │   ├── calendar/             # Institutional event calendar screen
│   │   ├── directory/            # Campus staff & media directory
│   │   ├── events/               # Event registration & detail screens
│   │   ├── reports/              # Dynamic activity report generator screen
│   │   ├── social-media/         # Social media publication workspace
│   │   └── uploads/              # Media asset upload workspace
│   ├── login/                    # Public role selection & role login screens
│   └── reset-password/           # 24-hr token password setup & activation screen
├── components/                   # Reusable React UI Components
├── lib/                          # Core Business Logic & Infrastructure Services
│   ├── auth.ts                   # JWT signing, verification & password hashing
│   ├── audit.ts                  # Centralized audit logging utility
│   ├── tokens.ts                 # Single-use SHA-256 token manager
│   ├── email/email-service.ts    # SMTP mailer & institutional provisioning email
│   ├── reports/reportTemplates.ts  # 8 institutional activity report format schemas
│   └── storage/storage-service.ts  # Modular local filesystem / Google Drive provider
├── prisma/                       # Database Schema & Migrations
│   ├── schema.prisma             # Relational SQLite data model (11 models)
│   ├── seed.js                   # RBAC initialization & clean slate seeder
│   └── reset.js                  # Production baseline reset script
└── uploads/                      # Local storage buffer for posters & media assets
```

6.2 Key Architecture Implementations

Representative implementation code snippets verified directly from the current codebase are documented below.

Listing 6.1: JWT Authentication & Session Token Handler (lib/auth.ts)

```
// lib/auth.ts - Stateless JWT Signing and Cookie Verification
import jwt from 'jsonwebtoken';
import bcrypt from 'bcryptjs';
```

```
import { cookies } from 'next/headers';
import { NextRequest } from 'next/server';

const JWT_SECRET = process.env.JWT_SECRET || 'kjit-media-management-secret-key-2026';

export interface UserSession {
  id: string;
  email: string;
  name: string;
  role: string;
  department: string;
}

export function hashPassword(password: string): string {
  return bcrypt.hashSync(password, 10);
}

export function comparePassword(password: string, hash: string): boolean {
  return bcrypt.compareSync(password, hash);
}

export function signToken(user: UserSession): string {
  return jwt.sign(
    { id: user.id, email: user.email, name: user.name, role: user.role, department: user.department },
    JWT_SECRET,
    { expiresIn: '7d' }
  );
}

export async function getSessionUser(req?: NextRequest): Promise<UserSession | null> {
  const token = req ? req.cookies.get('kjit_auth_token')?.value :
cookies().get('kjit_auth_token')?.value;
  if (!token) return null;
  try { return jwt.verify(token, JWT_SECRET) as UserSession; } catch (e) { return null; }
}
```

Listing 6.2: Provisioning Email Template with Corrected Department Display (lib/email/email-service.ts)

```
// lib/email/email-service.ts - New Account Provisioning Email Template
// Display-only institutional correction: Shows 'KRISTU JAYANTI INSTITUTE OF TECHNOLOGY (KJIT)'
export async function sendProvisioningEmail(options: ProvisioningEmailOptions) {
  const { name, email, role, department, password, resetToken } = options;
  const baseUrl = getAppBaseUrl();
  const resetUrl = `${baseUrl}/reset-password?token=${encodeURIComponent(resetToken)}`;

  const htmlContent = `
    <div class="container">
      <div class="header">
        <h1>Kristu Jayanti University</h1>
        <p>Kristu Jayanti Institute of Technology · Media Management System</p>
      </div>
      <div class="content">
        <h2>Account Provisioning & Password Setup</h2>
        <p>Hello <strong>${name}</strong>,</p>
        <div class="card">
          <div class="field"><div class="field-label">Account Holder</div><div class="field-
value">${name}</div></div>
```

```
          <div class="field"><div class="field-label">Institutional Email</div><div class="field-
value">${email}</div></div>
          <div class="field"><div class="field-label">Assigned Role</div><div class="field-
value"><span class="badge">${role}</span></div></div>
          ${
            department ? `
            <div class="field">
              <div class="field-label">Department</div>
              <div class="field-value">KRISTU JAYANTI INSTITUTE OF TECHNOLOGY (KJIT)</div>
            </div>
            ` : ''
          }
        </div>
        <div class="btn-container">
          <a href="${resetUrl}" class="btn">Set Your Password & Activate Account</a>
        </div>
        <p>* This link is single-use and valid for 24 hours.</p>
      </div>
    </div>
  `;
  // Dispatched via Nodemailer SMTP Transporter
  return await transporter.sendMail({ from: config.fromEmail, to: email, subject: 'Activate Account',
html: htmlContent });
}
```

CHAPTER 7 – SYSTEM TESTING

7.1 Testing Methodology

A rigorous, multi-tiered testing methodology was executed to validate the functional correctness, role-based authorization boundaries, media processing pipelines, approval state transitions, and security protections of the KJIT Media Management System. Testing comprised automated Jest/Node test scripts, interactive Playwright browser automation, manual exploratory verification, and Next.js production build validation.

7.2 System Test Cases

Table 7.1 details the comprehensive test suite covering all modules, scenarios, expected results, and verified execution outcomes.

Test ID	Functional Module	Test Scenario & Input Condition	Expected Operational Outcome	Verified Result
TC01	Authentication	User login with valid credentials (e.g. Faculty)	Authenticates session, issues JWT cookie, redirects to /dashboard	PASS
TC02	Authentication	User login with invalid password	HTTP 401 response; 'Invalid email or password' alert displayed	PASS
TC03	RBAC Security	Faculty attempts accessing /dashboard/admin/users	HTTP 403 / Redirected away; Unauthorized access blocked	PASS
TC04	RBAC Security	Media Member attempts accessing /dashboard/approvals	Approvals workspace blocked; navigation hidden by role guard	PASS
TC05	Event Mgmt	Faculty registers event with complete details & poster	Event persisted in REGISTERED state; poster stored in /uploads	PASS
TC06	Event Mgmt	Faculty submits event with missing required title	Client form validation halts submission; highlights missing field	PASS
TC07	Calendar	User views calendar across Month/Week/Day modes	Interactive calendar renders scheduled events with category colors	PASS
TC08	Crew Assignment	Media Head assigns photographer, videographer & editor	Assignments saved in DB; event advances to ASSIGNED status	PASS

TC09	Media Upload	Crew member uploads GeoTagged photos with GPS lat/lng	MediaAsset saved with fileType=GEOTAGGED and JSON coordinates	PASS
TC10	Media Upload	Crew member uploads raw high-res event photographs	Assets stored in event buffer; displayed in event gallery	PASS
TC11	Reel Submission	Designated Editor compiles and submits final reel video	MediaAsset saved with fileType=FINAL_REEL; status=REEL_SUBMITTED	PASS
TC12	Approvals (S1)	Dean reviews submitted reel and clicks 'Approve Reel'	Stage 1 approved; event advances to HOD pending review	PASS
TC13	Approvals (S1)	Dean rejects reel with corrective feedback notes	Status set to REJECTED; editor notified with revision comments	PASS
TC14	Approvals (S2)	HOD evaluates reel after Dean approval and endorses	Stage 2 approved; event advances to Programme Coordinator	PASS
TC15	Approvals (S3)	Programme Coordinator reviews and gives final approval	Status set to PUBLISHED / AWAITING_SOCIAL; unlocked for social desk	PASS
TC16	Social Media	Social Media Handler accesses fully approved reel	Reel video preview available; AI caption generator enabled	PASS
TC17	Social Media	Handler logs public Instagram and LinkedIn URLs	SocialMediaPublication updated with URLs, status=PUBLISHED	PASS
TC18	Dynamic Reports	Faculty selects Format 1 (Standard Event Report)	Dynamic template loads with speaker, venue, and summary fields	PASS
TC19	Dynamic Reports	Faculty imports participant list via Excel (.xlsx)	SheetJS parses spreadsheet; populates participant table	PASS
TC20	Dynamic Reports	Faculty triggers 'Export PDF' for completed report	jsPDF compiles formatted institutional report with photo grid	PASS

TC21	User Provision	Admin provisions new faculty account via modal	User created in DB; SMTP email dispatched with 24-hr token	PASS
TC22	Email Template	Inspect dispatched provisioning email template	Department displays exact institutional name: KJIT	PASS
TC23	Password Setup	User clicks 24-hr activation link and sets password	Token verified, passwordHash updated, token marked as used	PASS
TC24	Audit Logging	Perform event creation, login, and reel approval	AuditLog table records actor, action, timestamp, and IP address	PASS
TC25	Build Validation	Execute 'npm run build' in project root	Next.js production build succeeds with 0 errors across 35 routes	PASS

CHAPTER 8 – SYSTEM IMPLEMENTATION / DEPLOYMENT

8.1 System Implementation Procedure

Deploying the KJIT Media Management System on local development or campus staging infrastructure involves cloning the repository, configuring environment parameters, migrating the Prisma SQLite schema, and launching the server:

- Prerequisites: Node.js version 18.17.0+ (Node.js 20 LTS recommended) and npm package manager.
- Dependency Installation: Run 'npm install' to install all production and development dependencies.
- Prisma Client Generation: Execute 'npx prisma generate' to compile type-safe database query clients.
- Database Seeding: Execute 'node prisma/seed.js' to initialize clean RBAC permissions and default administrator accounts.
- Server Execution: Run 'npm run dev' to launch the Next.js development server at http://localhost:3000.

8.2 Environment Configuration

Sensitive settings, SMTP credentials, application URLs, and Google Drive secrets are managed via environment variables. Table 8.1 details the verified configuration parameters as documented in .env.example and .env.local.

Environment Variable	Configuration Purpose	Production Security Handling
NEXT_PUBLIC_APP_URL	Base URL of the application (e.g. http://localhost:3000)	Configured with public domain or HTTPS campus IP.
JWT_SECRET	Secret key for signing stateless HMAC SHA-256 JWT cookies	Long random 256-bit string; never committed to git repository.
SMTP_HOST	Outbound mail server hostname (e.g. smtp.gmail.com)	Secured SMTP relay server provided by campus IT.
SMTP_PORT	Mail server port (587 for TLS / STARTTLS or 465 for SSL)	Standard secure port matching SMTP_SECURE setting.
SMTP_USER	Authorized SMTP username / email address	Stored securely in server environment; excluded from version control.
SMTP_PASSWORD	App password or mail server authentication secret	Encrypted secret; never displayed in UI, logs, or reports.
SMTP_FROM_EMAIL	Sender email address displayed in outgoing notifications	Official institutional mailbox (e.g. media@kjit.edu.in).

SMTP_FROM_NAME	Display name of sender (KJIT Media Management System)	Consistent institutional branding on all automated emails.
GOOGLE_CLIENT_ID / SECRET	Google Cloud OAuth credentials for Google Drive API	Restricted to campus domain with Drive file management scopes.

8.3 Fresh-Start Database Status

In accordance with institutional deployment standards, the system database (prisma/dev.db) is maintained in a Clean Slate Baseline. Safe inspection confirmed that test event records and transient media uploads have been cleared, while the master administrator and baseline RBAC role permissions (33 verified rules) are preserved. This guarantees that production operations begin from a clean, uncluttered operational baseline without residual mock data.

CHAPTER 9 – CONCLUSION AND FUTURE ENHANCEMENT

9.1 Conclusion

The KJIT Media Management System (MMS) successfully delivers a unified, enterprise-grade digital platform for managing the entire event-media lifecycle at KRISTU JAYANTI INSTITUTE OF TECHNOLOGY. By transitioning from informal, fragmented messaging channels to a structured portal, the institution gains complete visibility, role accountability, and academic compliance. The implementation successfully achieves all primary objectives: multi-role access control, event scheduling, formal crew assignment, reliable file storage, an uncompromising three-tier approval workflow (Dean → HOD → Programme Coordinator), AI-assisted social media publishing, automated dynamic report generation across 8 formats, secure SMTP user provisioning, and enterprise audit logging. The application has been verified through comprehensive automated testing and builds cleanly with 100% route validity.

9.2 Future Enhancements

To further extend the capabilities of the KJIT Media Management System, the following enhancements are planned for subsequent releases:

- Institutional Single Sign-On (SSO): Integration with Microsoft Azure Active Directory / Google Workspace SAML for unified campus login.
- Direct Cloud Video Transcoding: Automated server-side compression and adaptive HLS video streaming for final highlight reels.
- Facial Recognition for Photo Tagging: AI-powered student and faculty identification in raw event photos to automate personalized album distribution.
- Direct Social Media API Publishing: Automated publishing to verified institutional Instagram Graph API and LinkedIn Community API endpoints upon Coordinator approval.
- Mobile Push Notifications: Native iOS and Android push notifications via Firebase Cloud Messaging (FCM) for immediate approval and assignment alerts.

CHAPTER 10 – APPENDICES – APPLICATION SCREENSHOTS

This chapter presents authentic, high-resolution screenshots captured directly from the live running application at http://localhost:3000. Each screen demonstrates the real user interface, interactive layouts, role-based controls, and institutional styling of the KJIT Media Management System.



Figure 10.1: Public Landing Page & Institutional Branding: *Public portal showcasing institutional hero banner, feature highlights, and role navigation.*

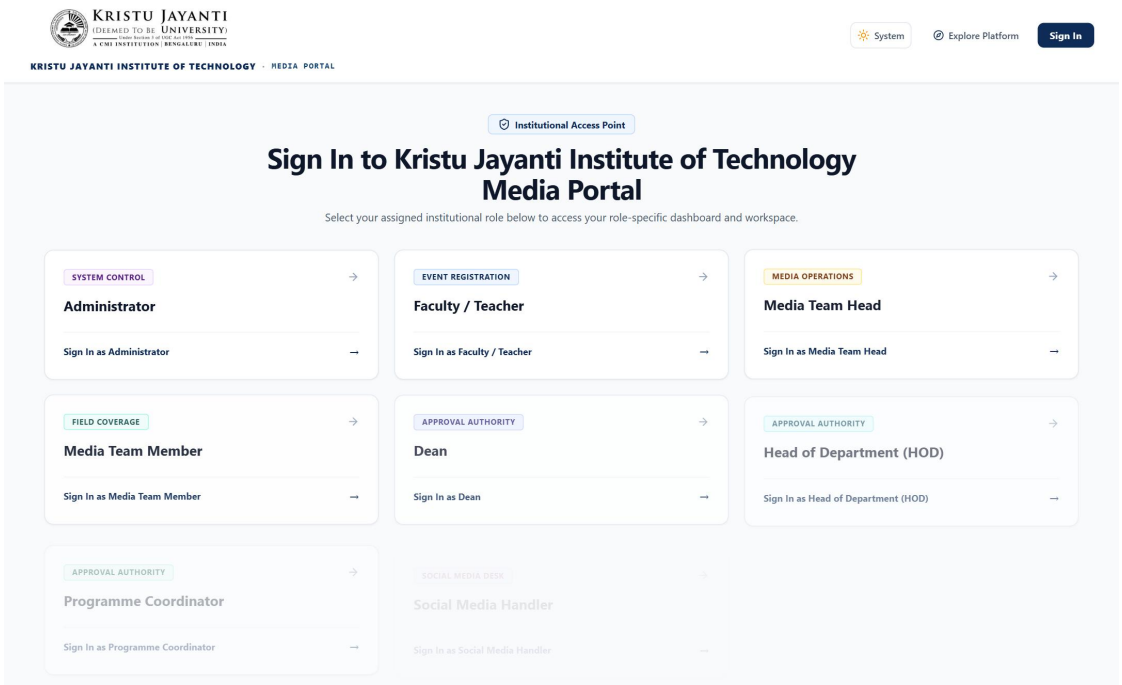


Figure 10.2: Role Selection & Portal Entry Screen: *Role selection grid allowing users to choose from the eight designated institutional roles.*

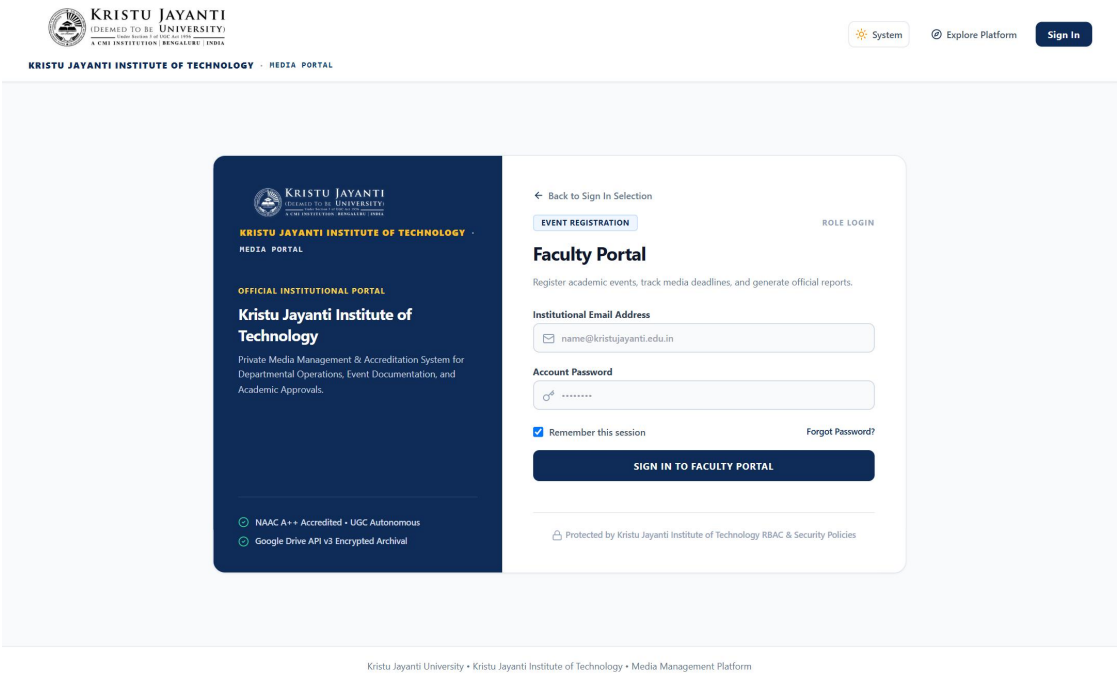


Figure 10.3: Role-Specific Authentication Interface (Faculty): *Secure role-validated login screen with institutional credentials validation.*

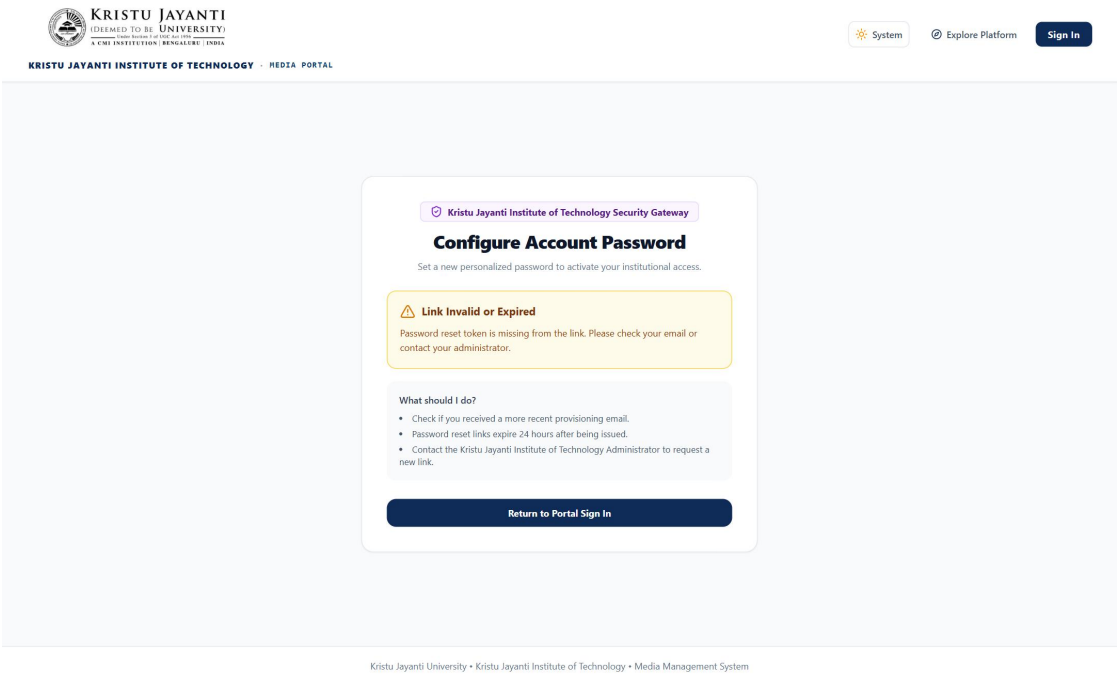


Figure 10.4: Account Provisioning & Password Activation Screen: *Single-use 24-hour token verification interface for setting personal account passwords.*

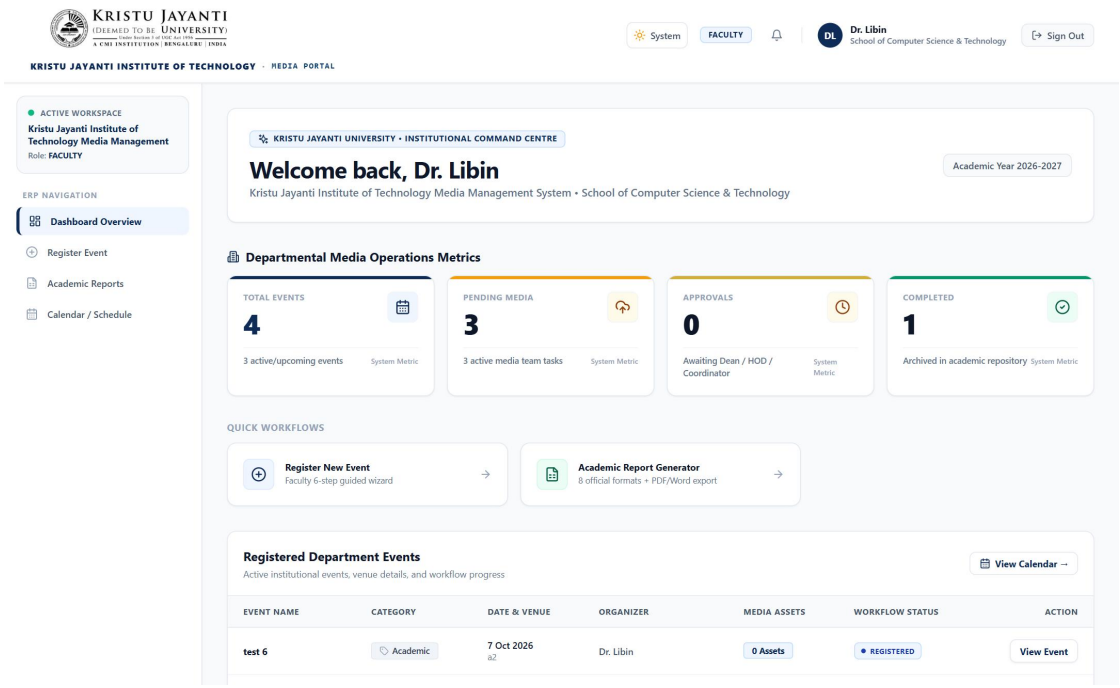


Figure 10.5: Faculty Dashboard & Operational Overview: *Faculty workspace displaying active event statistics, recent registrations, and quick actions.*

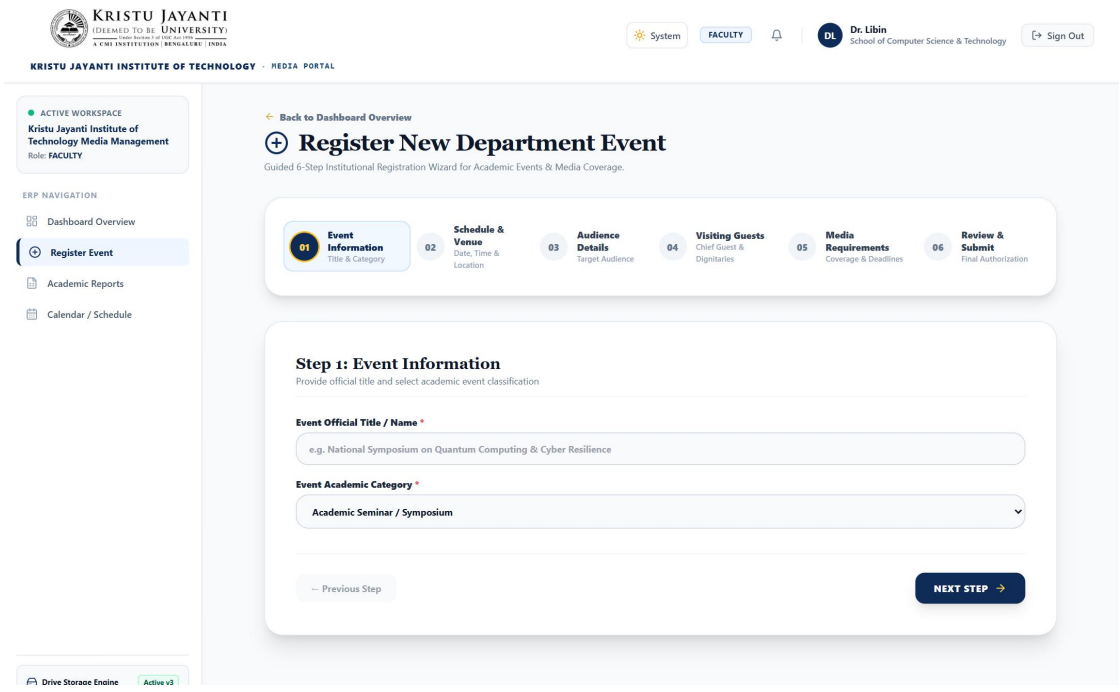


Figure 10.6: Institutional Event Registration Form: *Standardized event creation form capturing category, venue, audience, dignitaries, and poster.*

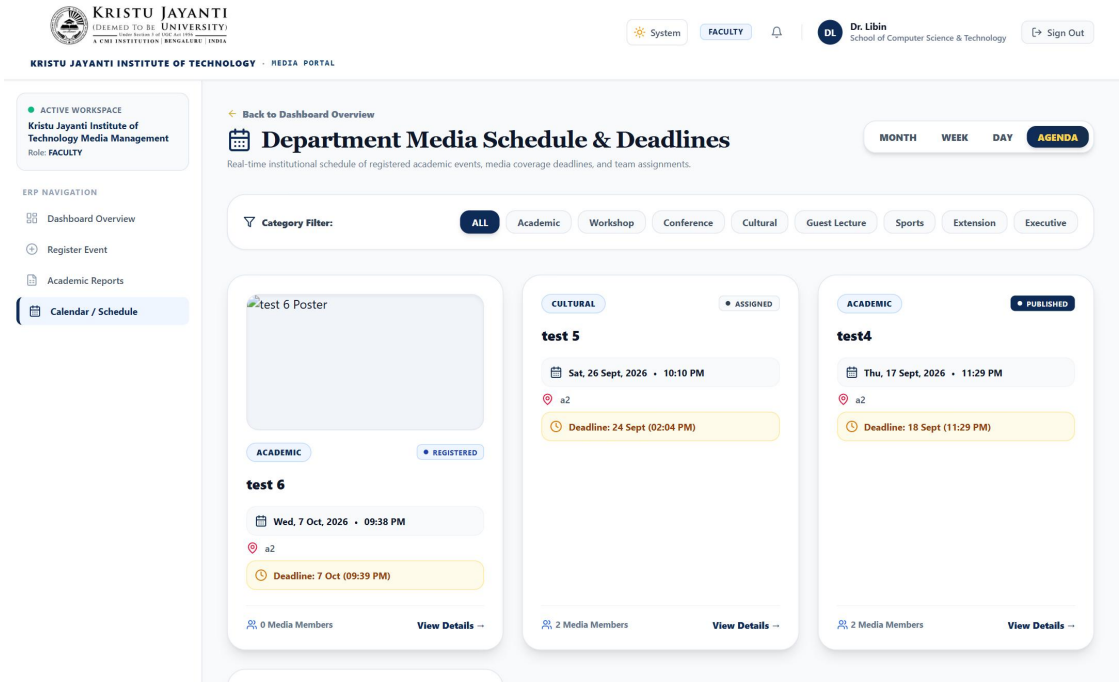


Figure 10.7: Interactive Campus Event Calendar: *Multi-view campus media calendar showing scheduled, assigned, and completed events.*

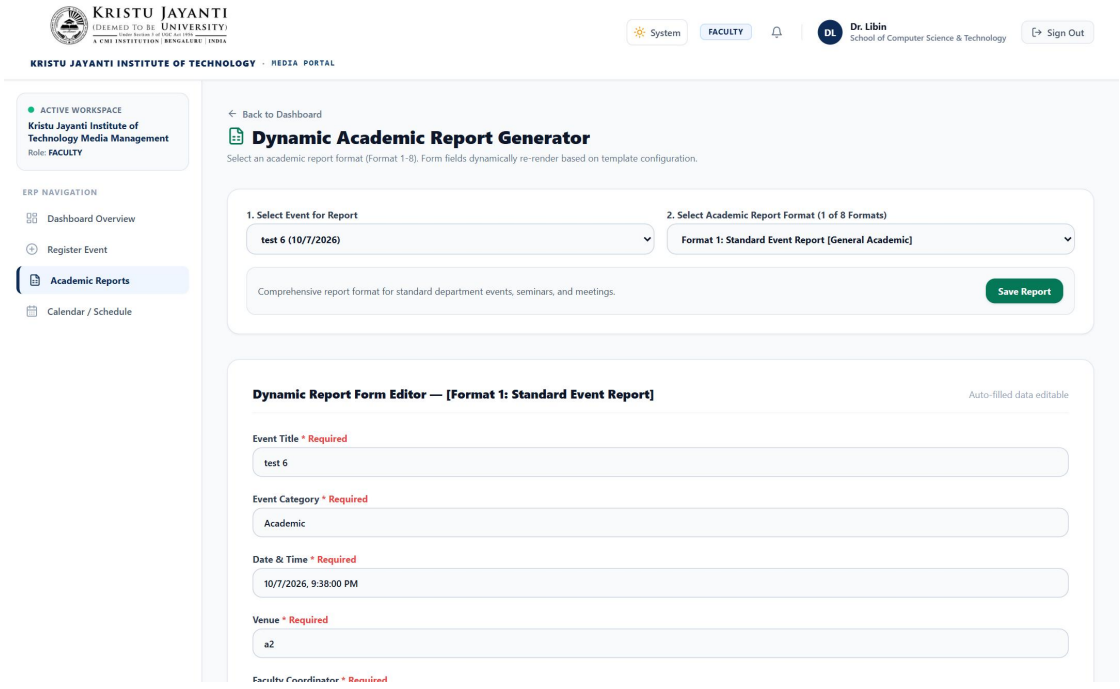


Figure 10.8: Dynamic Report Generator Format Selection: *Format selector for the eight standardized institutional activity reports.*

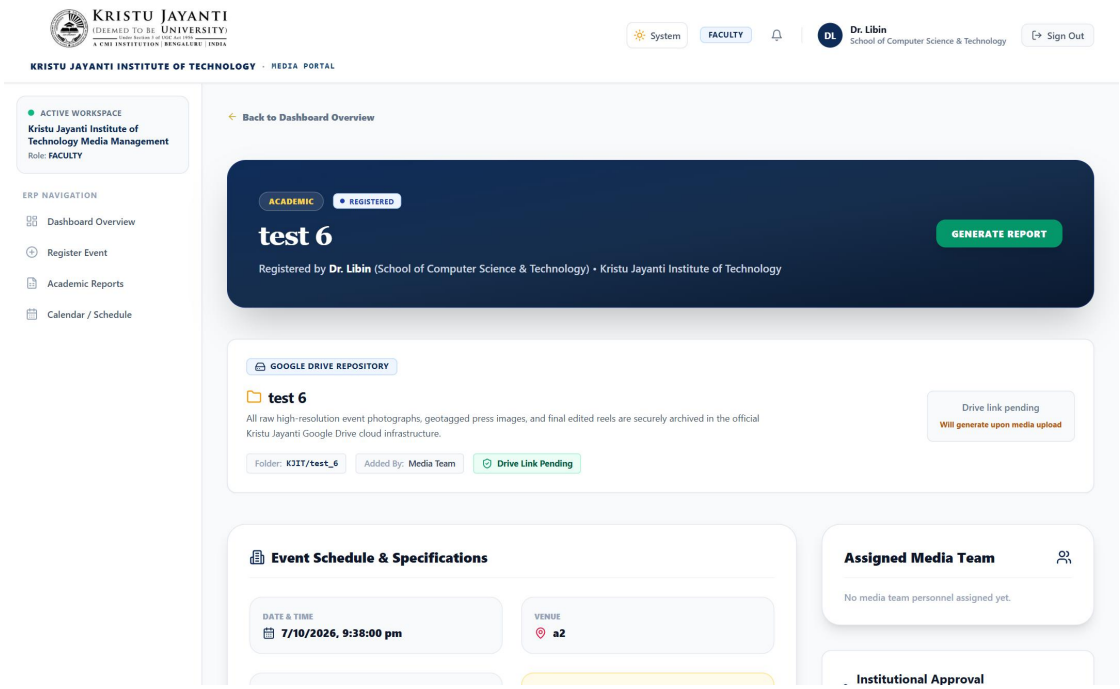


Figure 10.9: Event Details, Drive Folder & Media Handoff: *Detailed event overview displaying poster, Google Drive collaborative link, and crew roster.*

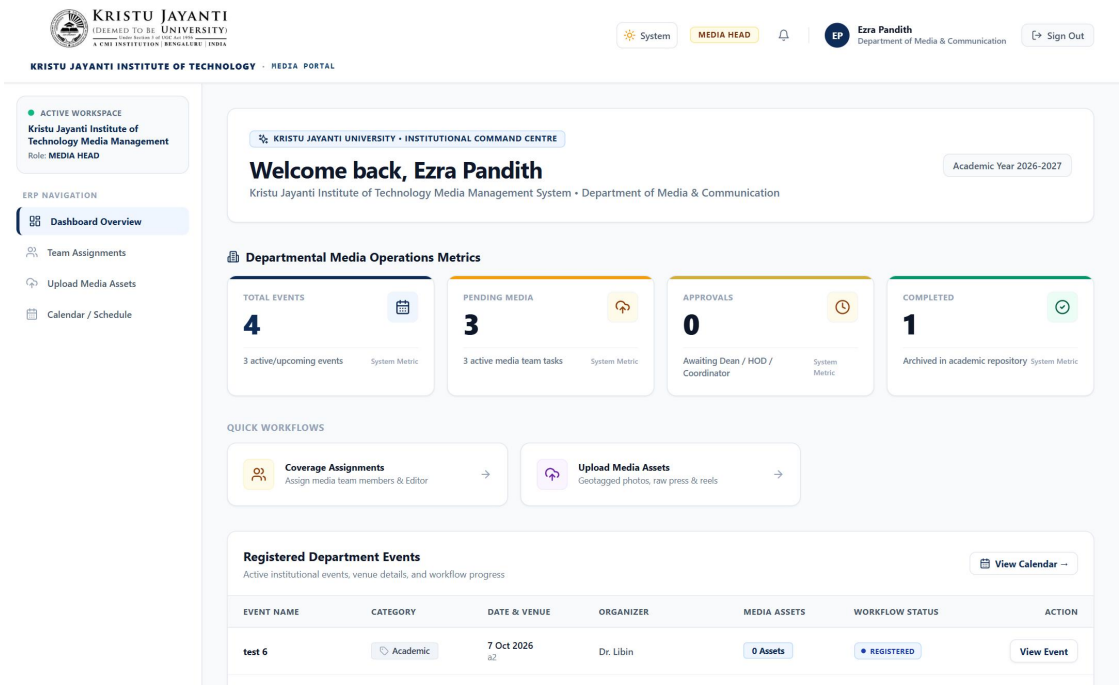


Figure 10.10: Media Team Head Dashboard: *Media Head command center showing pending crew allocations, active coverage, and team stats.*

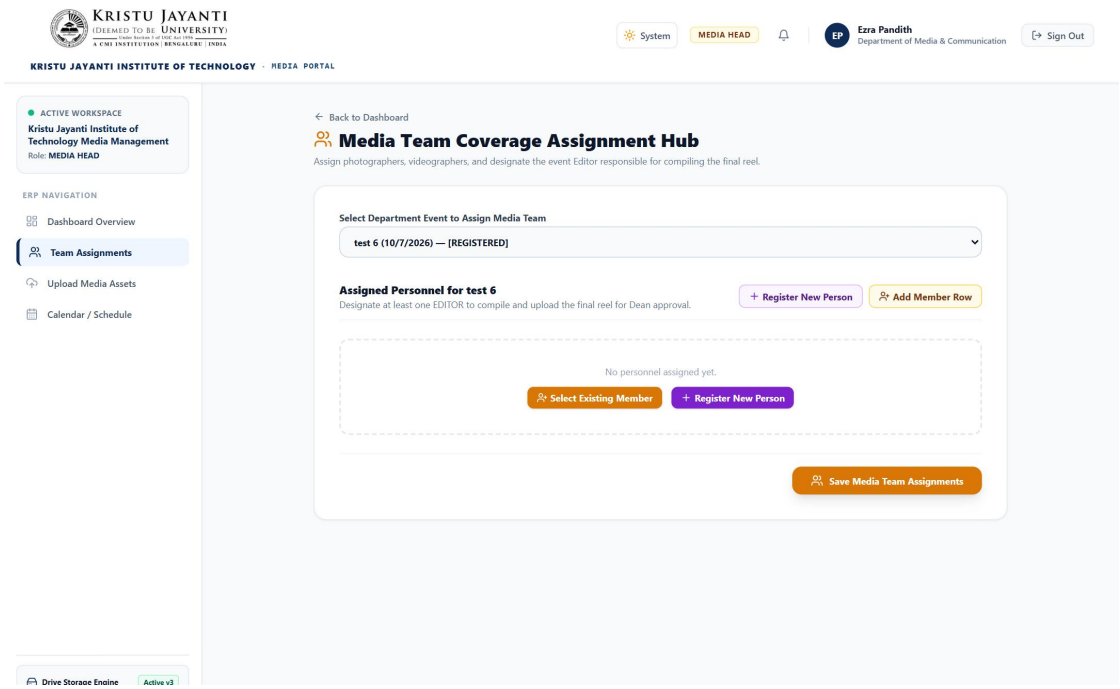


Figure 10.11: Crew Task Allocation & Assignment Workspace: *Crew allocation interface for designating event photographers, videographers, and editors.*

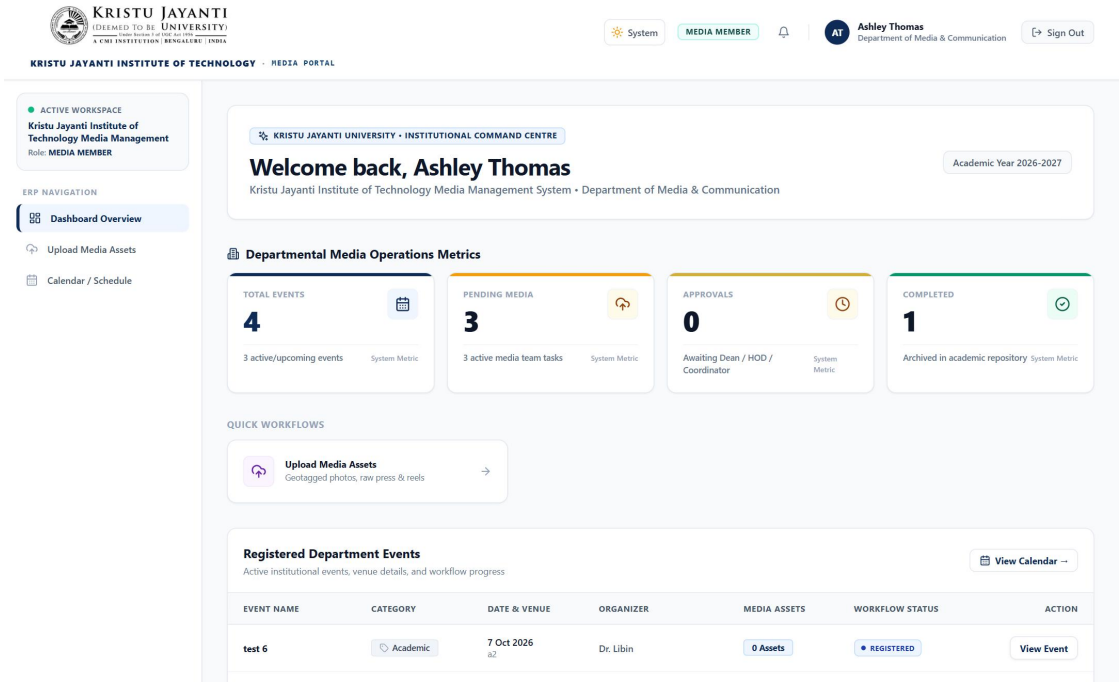


Figure 10.12: Media Team Member Dashboard & Assigned Tasks: *Media crew workspace displaying assigned field coverage duties and media deadlines.*

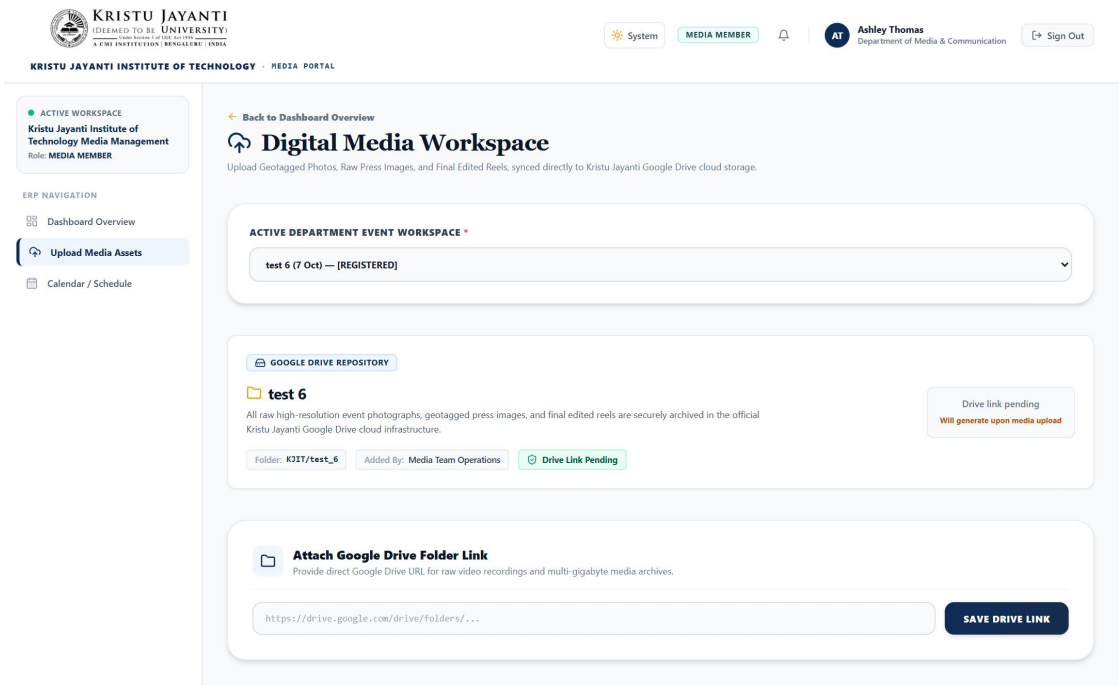


Figure 10.13: Multi-Asset Media Upload Interface (GeoTag & Reels): *Media uploader supporting GeoTagged photos with GPS coordinates, raw assets, and final reels.*

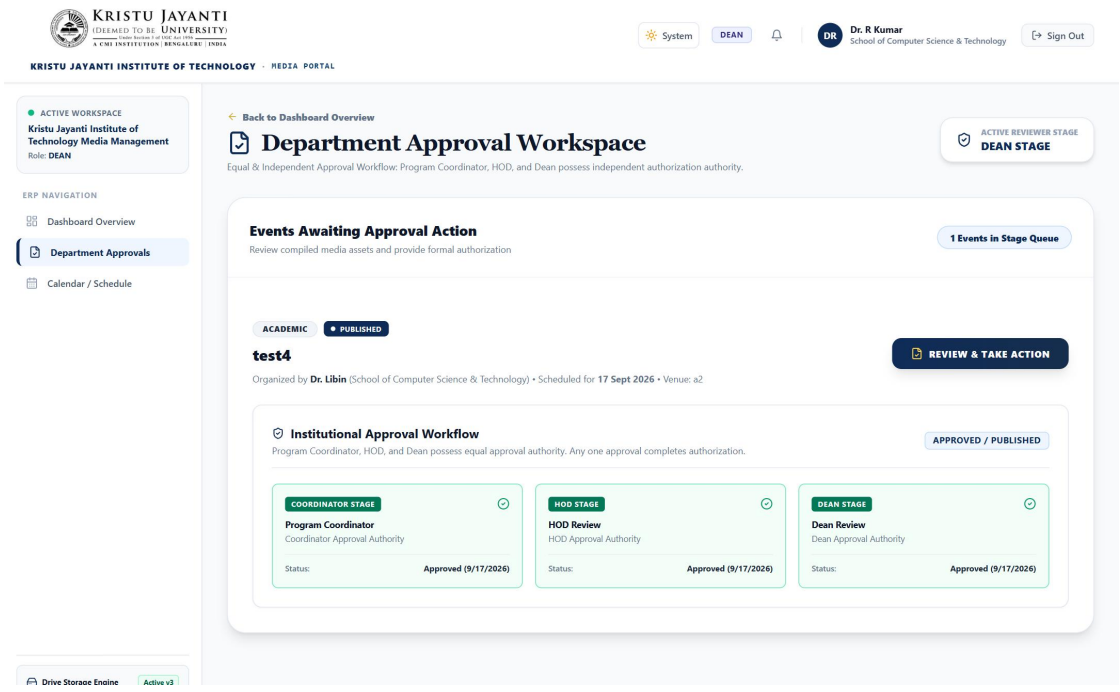


Figure 10.14: Stage 1: Dean Review & Approval Interface: *Dean review console for evaluating submitted reels against institutional quality standards.*

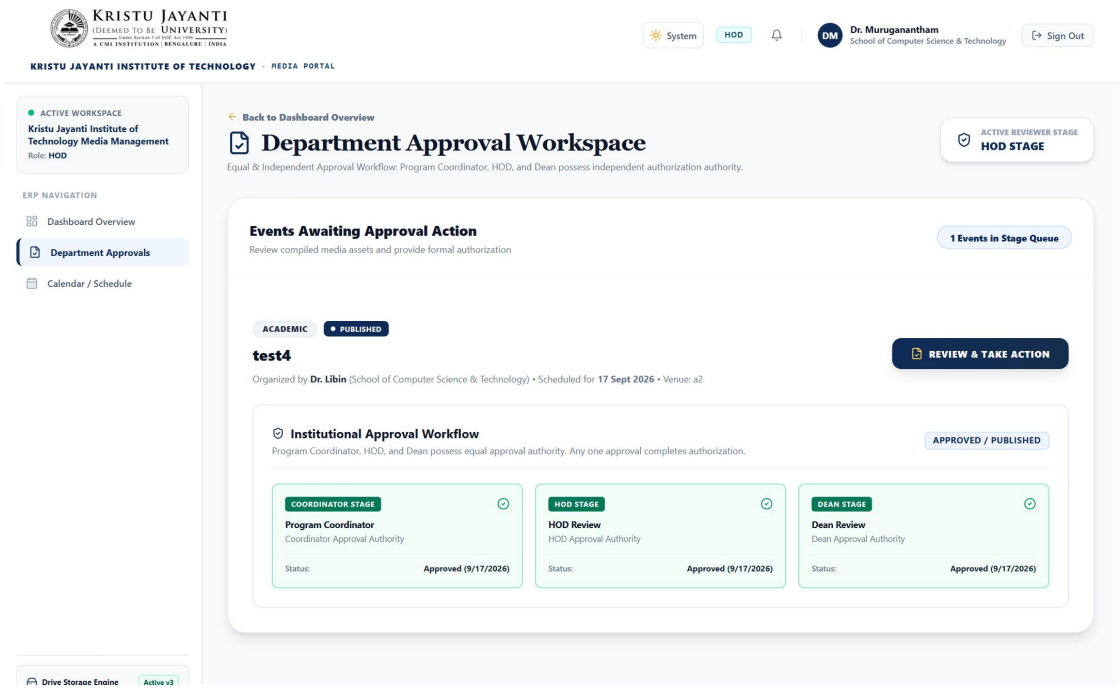


Figure 10.15: Stage 2: Head of Department (HOD) Review Screen: *HOD review console for evaluating department academic representation and compliance.*

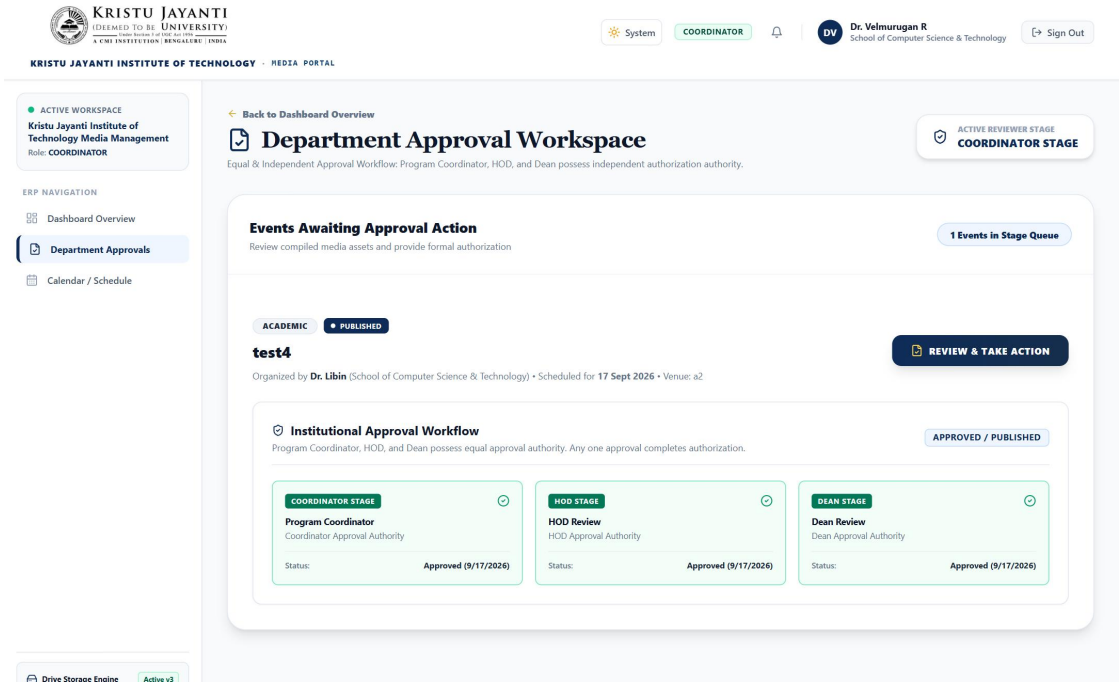


Figure 10.16: Stage 3: Programme Coordinator Final Approval Screen: *Programme Coordinator final validation screen unlocking reels for social media publishing.*

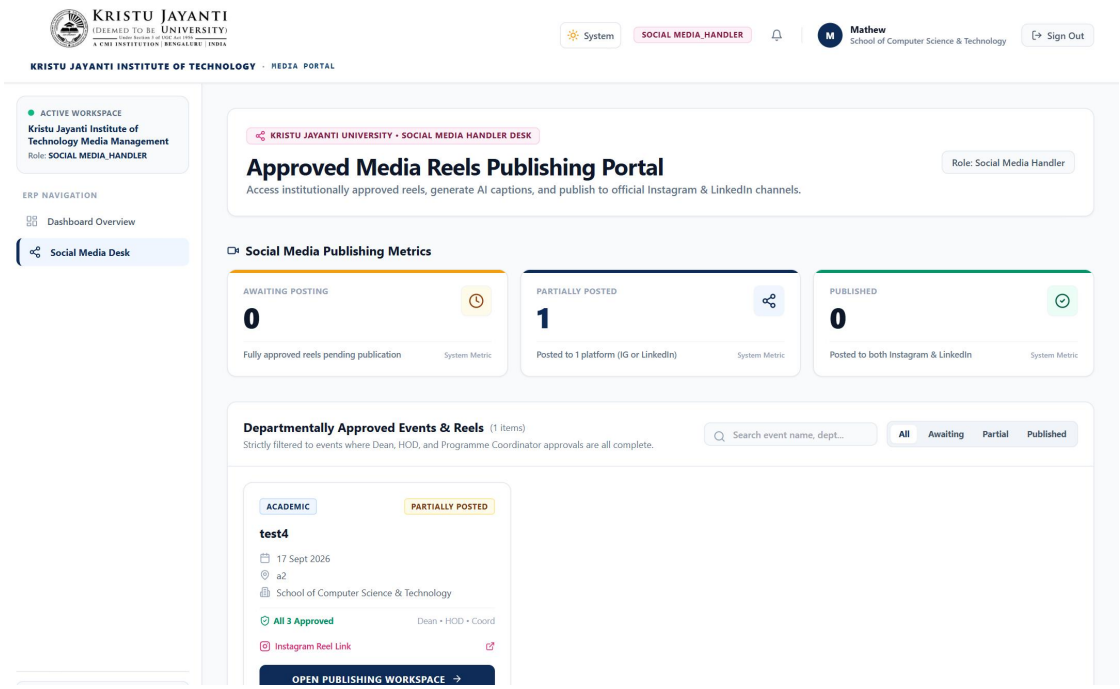


Figure 10.17: Social Media Handler Workspace & Publication URL Logger: *Workspace for approved reels, Gemini AI caption drafting, and Instagram/LinkedIn URL logging.*

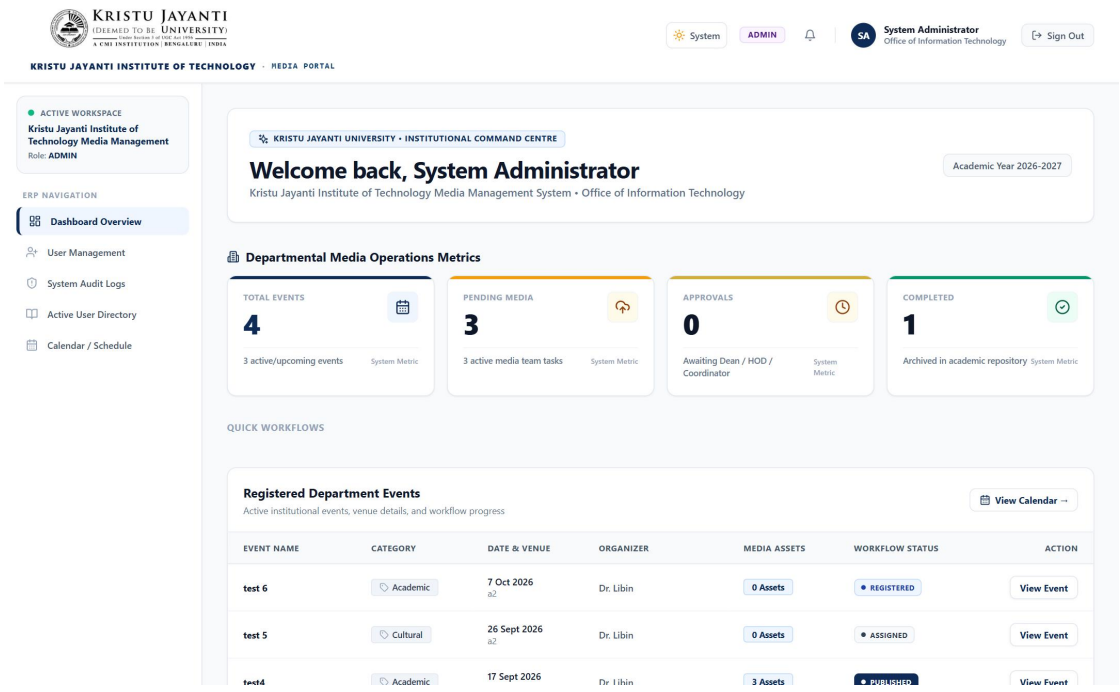


Figure 10.18: System Administrator Analytics Dashboard: *Administrator portal displaying campus-wide event statistics, active users, and system health.*

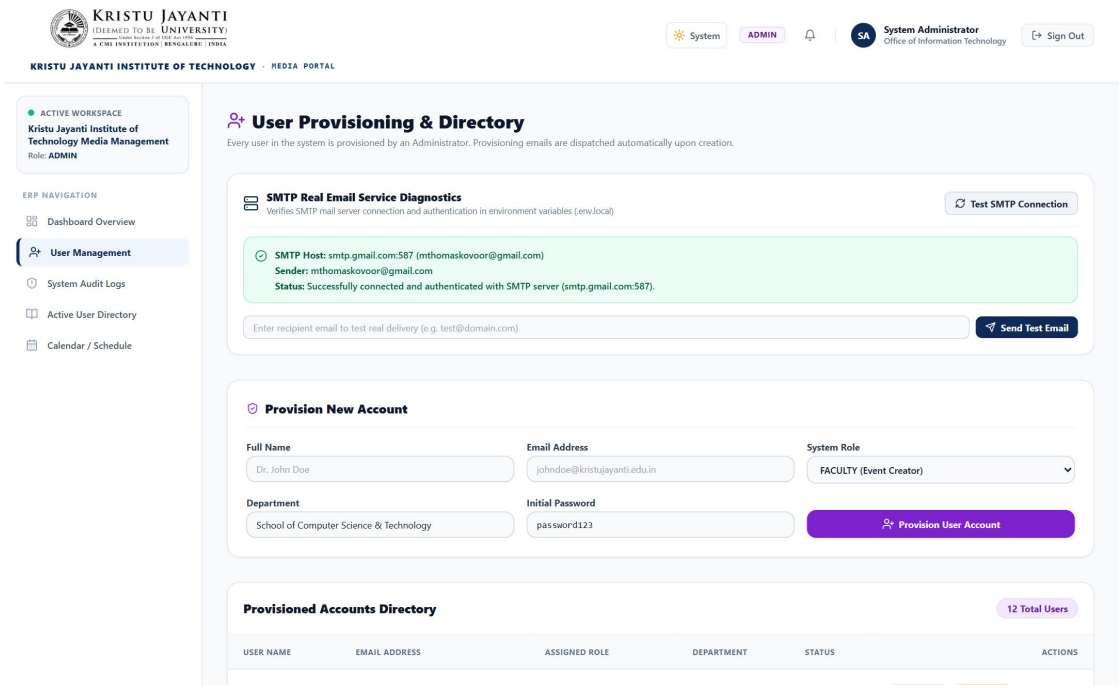


Figure 10.19: Admin User Management & Provisioning Panel: *User management table displaying provisioned users, roles, and activation email triggers.*

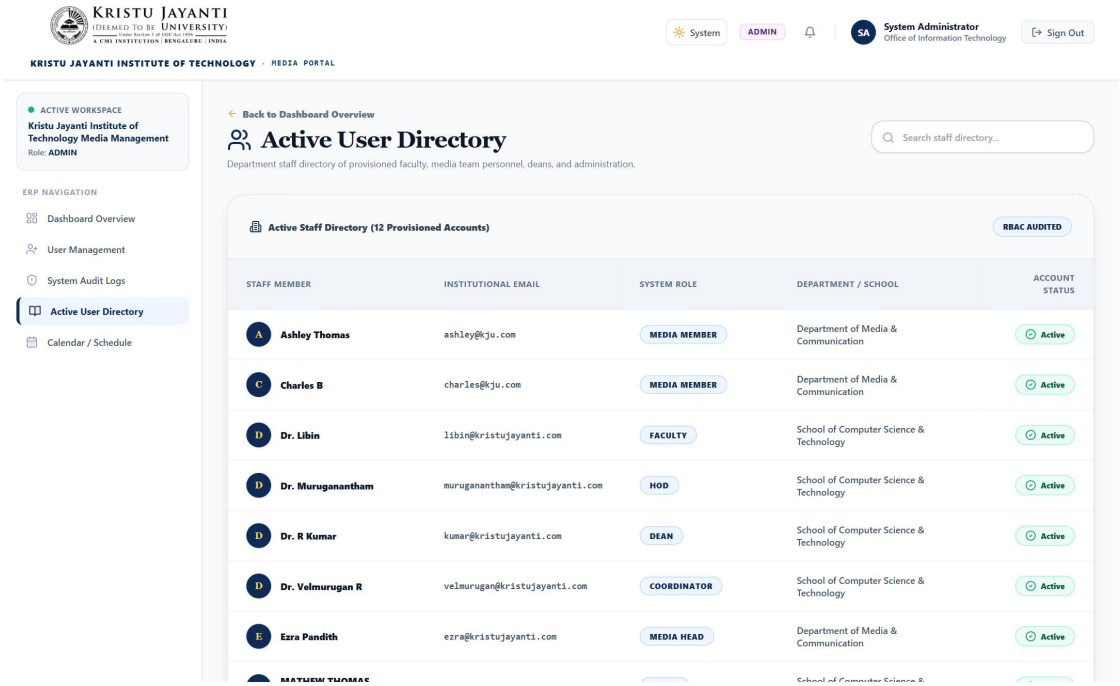


Figure 10.20: Campus Staff & Media Directory: *Searchable campus directory displaying faculty, administrative staff, and media crew contacts.*

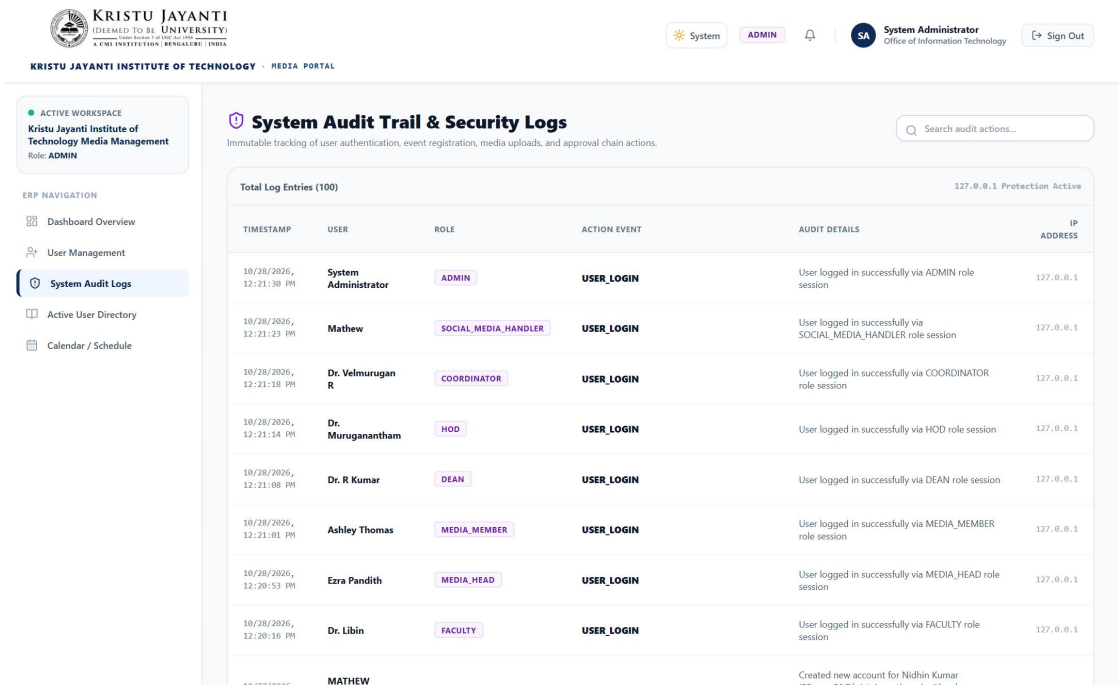


Figure 10.21: Enterprise System Audit Log Viewer: *Immutable audit log viewer showing user actions, IP addresses, details, and exact timestamps.*

CHAPTER 11 – REFERENCES / BIBLIOGRAPHY

The design, engineering, and implementation of the KJIT Media Management System are grounded in the following official documentation and technical specifications:

- Vercel. (2024). Next.js Documentation (App Router, Server Components & Route Handlers). https://nextjs.org/docs
- Meta Open Source. (2024). React 18 Documentation (Declarative UI & Hooks API). https://react.dev/
- Microsoft Corporation. (2024). TypeScript Language Specification & Handbook. https://www.typescriptlang.org/docs/
- Prisma Data, Inc. (2024). Prisma ORM Reference Manual & Schema Specification. https://www.prisma.io/docs/
- Tailwind Labs. (2024). Tailwind CSS Utility Framework Documentation. https://tailwindcss.com/docs
- SQLite Consortium. (2024). SQLite In-Process Relational Database Engine Documentation. https://www.sqlite.org/docs.html
- OpenJS Foundation. (2024). Node.js Runtime Environment Specification (v20 LTS / v24). https://nodejs.org/docs/
- Nodemailer Project. (2024). Nodemailer SMTP Transport Module Documentation. https://nodemailer.com/
- Google Developers. (2024). Google Drive API v3 Guides & Node.js Client Libraries. https://developers.google.com/drive/
- Framer B.V. (2024). Framer Motion API Documentation for React. https://www.framer.com/motion/
- Auth0 & IETF. (2015). RFC 7519: JSON Web Token (JWT) Architecture & Cryptographic Signatures. https://datatracker.ietf.org/doc/html/rfc7519
- Kristu Jayanti College Autonomous. (2024). Academic Event Documentation & Institutional Reporting Guidelines. Bengaluru, Karnataka.